

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN
NHÓM CHUYÊN MÔN KHOA HỌC MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN & ĐỒ ÁN

HỌC MÁY NÂNG CAO

**Đề tài: Xây dựng hệ thống chatbot hỗ trợ tra cứu
thông tin từ tài liệu bằng kỹ thuật RAG
(Retrieval-Augmented Generation)**

Giảng viên hướng dẫn: ThS. Nguyễn Đình Quý

Sinh viên thực hiện:
Phạm Hồng Thái - 0127067 - 67CS1
Lã Minh Khánh - 4004267 - 67CS1
Trịnh Quỳnh Anh - 0279367 - 67CS1
Nguyễn Hải Cường - 0174067 - 67CS1

Hà Nội, ngày 23 tháng 12 năm 2025

MỤC LỤC

LỜI NÓI ĐẦU	3
1 Giới thiệu tổng quan	4
1.1 Đặt vấn đề và tính cấp thiết của đề tài	4
1.1.1. Đặt vấn đề và tính cấp thiết của đề tài	4
1.2 Mục tiêu nghiên cứu	5
1.3 Phạm vi và đối tượng nghiên cứu	5
1.4 Cấu trúc báo cáo	5
2 Cơ sở lý thuyết	7
2.1 Tổng quan về Retrieval-Augmented Generation (RAG)	7
2.1.1. Sự ra đời và vai trò của Mô hình ngôn ngữ lớn (LLM)	7
2.1.2. RAG là gì?	8
2.1.3. Lý do chọn RAG	10
2.2 Các kỹ thuật tìm kiếm thông tin	10
2.2.1. Tìm kiếm từ khóa (Keyword Search/BM25)	10
2.2.2. Tìm kiếm ngữ nghĩa	12
2.2.3. Tìm kiếm lai	13
2.3 Mô hình xếp hạng lại	14
2.3.1. Kiến trúc Bi-Encoder và Cross-Encoder	15
2.3.2. Mô hình PhoRanker và ứng dụng cho tiếng Việt	16
2.4 Kỹ thuật mở rộng truy vấn	18
2.4.1. Kỹ thuật Sinh truy vấn đa dạng (Multi-Query/Generated Queries)	18
2.4.2. Kỹ thuật Nhúng tài liệu giả định (Hypothetical Document Embeddings - HyDE)	19
2.4.3. Mối quan hệ giữa Multi-Query và HyDE)	19
2.5 Các chỉ số đánh giá hệ thống	20
2.5.1. Chỉ số đánh giá giai đoạn Truy xuất (Retrieval Metrics)	20
2.5.2. Chỉ số đánh giá giai đoạn Sinh câu trả lời (Generation Metrics)	21
3 Phương pháp đề xuất	23
3.1 Kiến trúc hệ thống tổng quát	23
3.1.1. Luồng đánh chỉ mục dữ liệu (Offline Indexing Pipeline)	23
3.1.2. Luồng truy vấn và sinh câu trả lời (Real-time Inference Pipeline)	24
3.2 Quy trình xử lý dữ liệu	25
3.2.1. Tiền xử lý và làm sạch dữ liệu	25

3.2.2. Chiến lược phân đoạn văn bản (Chunking Strategy)	27
3.3 Module truy vấn nâng cao	28
3.3.1. Cài đặt Query Expansion: Kỹ thuật HyDE (Hypothetical Document Embeddings)	28
3.3.2. Cơ chế Tìm kiếm lai (Hybrid Search Mechanism)	30
3.4 Chiến lược Fine-tuning mô hình Reranking	31
3.4.1. Lựa chọn Base Model (itdainb/PhoRanker)	32
3.4.2. Phương pháp xây dựng dữ liệu huấn luyện	33
3.4.3. Hàm mất mát (Loss Function) và chiến lược huấn luyện . . .	34
3.5 Module sinh câu trả lời	37
3.5.1. Kiến trúc luồng xử lý dữ liệu (Generation Pipeline)	37
3.5.2. Chiến lược Truy hồi nâng cao (Advanced Retrieval Strategy) .	39
3.5.3. Tối ưu hóa Reranking và Mở rộng ngữ cảnh (Reranking & Context Optimization)	41
3.5.4. Kỹ thuật Prompt Engineering và Cơ chế An toàn (Prompting & Safety)	42
4 Thực nghiệm và cài đặt	44
4.1 Môi trường thực nghiệm và công nghệ sử dụng	44
4.2 Quy trình xử lý dữ liệu	44
4.2.1. Dữ liệu gốc (Tài liệu môn học)	44
4.2.2. Xây dựng bộ dữ liệu đánh giá chuẩn (Golden Dataset/QA_Gold)	45
4.3 Quá trình Fine-tuning PhoRanker	46
4.3.1. Thiết lập tham số huấn luyện (Hyperparameters)	46
4.3.2. Biểu đồ huấn luyện và Giám sát hiệu năng	47
4.4 Demo minh họa	49
4.5 Kết quả thực nghiệm	50
4.4.1. Đánh giá khả năng Truy xuất (Retrieval Performance)	50
4.4.2. Đánh giá khả năng Sinh văn bản (Generation Quality)	51
4.4.3. Đánh giá định tính (LLM-as-a-Judge)	51
4.4.4. Phân tích trích dẫn	52
5 Kết luận và hướng phát triển	53
5.1 Kết luận	53
5.2 Hạn chế của đề tài	53
5.3 Hướng phát triển trong tương lai	54
Tài liệu tham khảo	56

LỜI NÓI ĐẦU

Chúng em xin gửi lời cảm ơn sâu sắc đến thầy Nguyễn Đình Quý, người đã tận tâm giảng dạy và truyền đạt những kiến thức quý báu trong suốt quá trình học tập học phần "Học máy nâng cao". Nhờ sự hướng dẫn nhiệt tình và tận tâm của thầy, chúng em đã có được nền tảng kiến thức vững chắc để bước đầu khám phá và tiếp cận sâu hơn với chuyên ngành, cũng như tự tin thực hiện các nghiên cứu và sáng tạo trong tương lai.

Chúng em cũng xin chân thành cảm ơn thầy vì đã không ngừng hỗ trợ và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài này. Những lời chỉ bảo và góp ý của thầy chính là nguồn động lực lớn giúp chúng em hoàn thành tốt nhiệm vụ được giao.

Bên cạnh đó, chúng em xin gửi lời tri ân đến gia đình, bạn bè và các anh chị tiền bối đã luôn động viên và cung cấp cho chúng em những tài liệu, thông tin hữu ích trong suốt quá trình thực hiện đề tài.

Dù đã nỗ lực hết mình và làm việc với tinh thần trách nhiệm cao, nhưng chắc chắn bài làm của chúng em vẫn không thể tránh khỏi những sai sót. Chúng em rất mong nhận được sự đóng góp ý kiến từ các thầy cô để bài tập được hoàn thiện hơn, đồng thời giúp chúng em tích lũy thêm kinh nghiệm quý báu cho những lần nghiên cứu sau.

Chúng em xin chân thành cảm ơn!

Ngày 23 tháng 12 năm 2025

Nhóm 13

TRỊNH QUỲNH ANH

LÃ MINH KHÁNH

PHẠM HỒNG THÁI

NGUYỄN HẢI CƯỜNG

Chương 1: Giới thiệu tổng quan

Chương này đóng vai trò là phần mở đầu quan trọng, cung cấp cái nhìn toàn diện về bối cảnh nghiên cứu và lý do dẫn đến việc hình thành đề tài. Trong kỷ nguyên trí tuệ nhân tạo (AI) bùng nổ, việc khai thác sức mạnh của các mô hình ngôn ngữ lớn (Large Language Models - LLMs) để quản trị tri thức đã trở thành một xu thế tất yếu. Tuy nhiên, việc ứng dụng AI vào các dữ liệu chuyên biệt đòi hỏi sự chính xác và khả năng cập nhật liên tục – điều mà các mô hình truyền thống thường gặp khó khăn.

Nội dung của chương này sẽ tập trung làm rõ những thách thức trong việc tra cứu thông tin từ tài liệu nội bộ, từ đó đề xuất giải pháp sử dụng kỹ thuật RAG (Retrieval-Augmented Generation). Bên cạnh đó, chương 1 cũng xác định rõ ràng mục tiêu cần đạt được, đối tượng dữ liệu chuyên ngành mà hệ thống hướng tới, cũng như lộ trình triển khai báo cáo qua các chương tiếp theo. Đây là nền tảng lý luận và thực tiễn để xây dựng một hệ thống chatbot thông minh, tin cậy và có giá trị ứng dụng cao.

1.1 Đặt vấn đề và tính cấp thiết của đề tài

Trong bối cảnh chuyển đổi số mạnh mẽ, lượng tài liệu số hóa dưới dạng văn bản (PDF, Docx, Slide) trong các tổ chức và cơ sở giáo dục đang gia tăng theo cấp số nhân. Việc tra cứu thông tin thủ công trong các tài liệu chuyên ngành dài hàng trăm trang trở thành một rào cản lớn, gây lãng phí thời gian và giảm hiệu suất làm việc, học tập.

Sự ra đời của các mô hình ngôn ngữ lớn (Large Language Models - LLMs) như GPT, Gemini hay Llama đã mở ra khả năng tương tác tự nhiên giữa người và máy. Tuy nhiên, khi áp dụng vào việc tra cứu tài liệu chuyên biệt, các mô hình này bộc lộ hai nhược điểm cốt lõi:

- **Hiện tượng ảo giác (Hallucination):** Mô hình có xu hướng tự tạo ra thông tin nghe có vẻ thuyết phục nhưng thực chất không có trong dữ liệu gốc.
- **Thiếu hụt tri thức chuyên sâu:** Các LLM được huấn luyện trên dữ liệu công khai, do đó không có khả năng truy cập vào các nguồn tài liệu nội bộ, giáo trình chuyên ngành mới cập nhật hoặc các tài liệu có tính chất riêng tư.

Kỹ thuật **RAG (Retrieval-Augmented Generation)** nổi lên như một giải pháp tối ưu nhằm kết hợp sức mạnh lập luận của LLM với khả năng truy xuất dữ liệu từ các nguồn tin cậy bên ngoài. Bằng cách nhúng (embedding) và truy xuất các đoạn văn bản có liên quan trước khi tạo phản hồi, RAG giúp chatbot cung cấp câu trả

lời chính xác, có dẫn chứng và giảm thiểu tối đa hiện tượng ảo giác. Vì vậy, việc thực hiện đề tài "**Xây dựng hệ thống chatbot hỗ trợ tra cứu thông tin từ tài liệu bằng kỹ thuật RAG**" mang ý nghĩa cấp thiết cả về mặt kỹ thuật lẫn thực tiễn ứng dụng.

1.2 Mục tiêu nghiên cứu

- **Mục tiêu tổng quát:** Xây dựng một hệ thống chatbot thông minh hỗ trợ người dùng truy xuất và giải đáp thông tin chính xác từ tập hợp các tài liệu chuyên ngành dựa trên kiến trúc RAG.
- **Mục tiêu cụ thể:**
 - Nghiên cứu quy trình xử lý ngôn ngữ tự nhiên (NLP), kỹ thuật chuyển đổi văn bản thành vector (Embedding) và lưu trữ dữ liệu vector.
 - Xây dựng mô hình truy xuất (Retrieval) tối ưu để tìm kiếm chính xác ngữ cảnh từ kho dữ liệu chuyên ngành.
 - Tích hợp mô hình ngôn ngữ lớn để tổng hợp câu trả lời từ dữ liệu đã truy xuất.
 - Thiết kế và triển khai giao diện người dùng (User Interface) thân thiện, hỗ trợ tương tác đa phương thức.

1.3 Phạm vi và đối tượng nghiên cứu

- **Đối tượng nghiên cứu:** Các phương pháp xử lý văn bản và phân đoạn dữ liệu (Chunking strategies).
- **Mục tiêu cụ thể:**
 - Các mô hình nhúng văn bản và hệ quản trị cơ sở dữ liệu vector (Vector Database).
 - Phạm vi dữ liệu chuyên ngành: Đề tài tập trung xử lý tập dữ liệu là các tài liệu thuộc chuyên ngành Khoa học máy tính, bao gồm: giáo trình, slide bài giảng, và các tài liệu khoa học liên quan.
 - Định dạng dữ liệu đầu vào: Ưu tiên các định dạng phổ biến như .pdf, .docx, .txt.
 - Ngôn ngữ hỗ trợ: Tiếng Việt.

1.4 Cấu trúc báo cáo

Nội dung của báo cáo được tổ chức thành 05 chương chính, trình bày logic từ bối cảnh, lý thuyết đến quy trình triển khai và đánh giá thực nghiệm:

- **CHƯƠNG 1. GIỚI THIỆU TỔNG QUAN:** Trình bày lý do lựa chọn đề tài,

tính cấp thiết của việc xây dựng hệ thống chatbot hỗ trợ tra cứu. Xác định mục tiêu nghiên cứu, phạm vi dữ liệu chuyên ngành và đối tượng mà đề tài hướng tới.

- **CHƯƠNG 2. CƠ SỞ LÝ THUYẾT:** Tập trung nghiên cứu các kiến thức nền tảng về Mô hình ngôn ngữ lớn (LLM), kỹ thuật Retrieval-Augmented Generation (RAG). Tìm hiểu về cơ chế hoạt động của Vector Database, các mô hình nhúng (Embedding) và các framework hỗ trợ xây dựng hệ thống AI.
- **CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT:** Mô tả chi tiết kiến trúc hệ thống Chatbot được đề xuất. Trình bày quy trình xử lý dữ liệu đầu vào (ETL), chiến lược phân đoạn văn bản (Chunking), quy trình truy xuất thông tin và cách thức thiết kế Prompt để tối ưu hóa câu trả lời từ LLM.
- **CHƯƠNG 4. THỰC NGHIỆM VÀ CÀI ĐẶT:** Trình bày chi tiết về môi trường phát triển (ngôn ngữ lập trình, thư viện), quá trình cài đặt mã nguồn và xây dựng giao diện người dùng. Mô tả các kịch bản thực nghiệm trên tập dữ liệu môn học chuyên ngành đã xác định, bao gồm quá trình Fine-tuning mô hình PhoRanker và đánh giá kết quả.
- **CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN:** Tổng kết các kết quả mà đề tài đã đạt được so với mục tiêu ban đầu. Phân tích ưu điểm và hạn chế của hệ thống, đồng thời đề xuất các hướng nghiên cứu mở rộng như tối ưu hóa quy trình Reranking hoặc tích hợp đa phương thức (Multimodal) để nâng cao hiệu quả hệ thống.

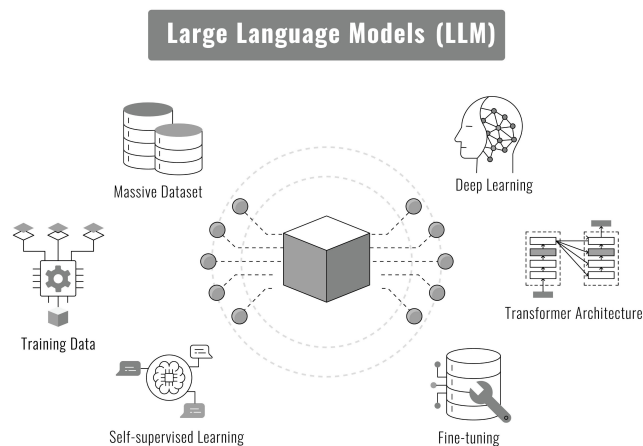
Chương 2: Cơ sở lý thuyết

2.1 Tổng quan về Retrieval-Augmented Generation (RAG)

2.1.1. Sự ra đời và vai trò của Mô hình ngôn ngữ lớn (LLM)

Mô hình ngôn ngữ lớn (LLM, Large Language Model), tiêu biểu là kiến trúc Transformer, đã tạo nên bước ngoặt trong xử lý ngôn ngữ tự nhiên (NLP). Các mô hình như GPT-4, Llama 3, hay Gemma được huấn luyện trên khối lượng dữ liệu khổng lồ (Petabytes), giúp chúng sở hữu "tri thức tham số" (Parametric Memory) cực kỳ mạnh mẽ. LLM có khả năng hiểu ngữ cảnh, suy luận logic và sinh văn bản tự nhiên vượt trội.

Các mô hình ngôn ngữ được huấn luyện trước quy mô lớn đã được chứng minh là có khả năng lưu trữ kiến thức thực tế trong các tham số của chúng và đạt được kết quả tiên tiến nhất khi được tinh chỉnh trên các tác vụ xử lý ngôn ngữ tự nhiên tiếp theo.



Hình 2.1: Ứng dụng của LLM.

Tuy nhiên, trong các ứng dụng thực tế đòi hỏi tính chính xác cao (như tài liệu chuyên ngành môn học), LLM thuần túy bộc lộ ba điểm yếu cốt lõi:

- Hiện tượng ảo tưởng (Hallucination): LLM có xu hướng tạo ra thông tin nghe có vẻ thuyết phục nhưng sai sự thật khi không có dữ liệu thực tế.
- Kiến thức bị đóng băng (Knowledge Cut-off): Mô hình chỉ biết những gì có trong dữ liệu huấn luyện tại thời điểm đó. Nó không thể truy cập thông tin mới phát sinh hoặc tài liệu nội bộ (ví dụ: giáo trình mới cập nhật của nhóm bạn).
- Thiếu khả năng kiểm chứng: LLM không thể chỉ ra nguồn gốc chính xác của

thông tin mà nó đưa ra.

2.1.2. RAG là gì?

Để giải quyết các hạn chế trên, Patrick Lewis và các cộng sự (từ Facebook AI Research) đã đề xuất khái niệm Retrieval-Augmented Generation (RAG) vào năm 2020.

RAG là một kiến trúc lai (Hybrid Architecture) kết hợp giữa:

1. Parametric Memory: Kiến thức đã được học và lưu trữ trong các trọng số của LLM.
2. Non-parametric Memory: Kho tri thức bên ngoài (ví dụ: các đoạn văn bản trong giáo trình môn học) được lưu dưới dạng Vector Database.

Trong mô hình học máy truyền thống (LLM "truyền thống" hay Base LLM), tri thức được lưu giữ hoàn toàn dưới dạng các trọng số (weights) sau quá trình tiền huấn luyện (Pre-training). Đây được gọi là Bộ nhớ tham số (Parametric Memory).

Tuy nhiên, RAG thay đổi mô thức này bằng cách giới thiệu Bộ nhớ phi tham số (Non-parametric Memory). Đây là một kho dữ liệu bên ngoài (thường là Vector Database) chứa các tài liệu văn bản thô. Thay vì bắt mô hình phải "ghi nhớ" mọi thứ, RAG cho phép mô hình "tra cứu" thông tin khi cần thiết, tương tự như việc một thí sinh làm bài thi theo hình thức "mở tài liệu" (Open-book) thay vì "thi đóng" (Closed-book).

Dưới góc độ xác suất, một LLM truyền thống dự đoán từ tiếp theo dựa trên xác suất:

$$P(y|x)$$

Trong đó x là câu truy vấn và y là câu trả lời.

Trong kiến trúc RAG, xác suất này được mở rộng thông qua một biến phụ z (đoạn văn bản được truy xuất). Theo bài báo gốc của Lewis và các cộng sự (2020) [1], cơ chế này được định nghĩa qua hai thành phần:

- Retriever (Mô hình truy xuất): Ký hiệu là $p_{\eta}(z|x)$, tính toán xác suất (hoặc độ tương đồng) của tài liệu z đối với câu hỏi x .
- Generator (Mô hình sinh): Ký hiệu là $p_{\theta}(y|x, z)$, sinh ra câu trả lời y dựa trên sự kết hợp của câu hỏi gốc x và tài liệu bổ trợ z .

Công thức tổng quát cho RAG (dưới dạng biên giải - Marginalization) là:

$$P(y|x) \approx \sum_{z \in \text{top-}k(R(x))} p_{\eta}(z|x) p_{\theta}(y|x, z)$$

Hệ thống sẽ lấy k đoạn văn bản có điểm số cao nhất từ bộ nhớ ngoài để làm ngữ cảnh đầu vào cho mô hình sinh.

RAG hoạt động giống như bất kỳ mô hình seq2seq (Sequence-to-Sequence)¹ nào khác [2]. Tuy nhiên, RAG có một thành phần trung gian giúp truy xuất các tài liệu ngữ cảnh từ một cơ sở tri thức bên ngoài (như kho ngữ liệu văn bản truyền thống và hiện đại). Sau đó, các tài liệu này được sử dụng kết hợp với chuỗi đầu vào và được chuyển đến bộ tạo seq2seq bên dưới.

Bước truy xuất thông tin này cho phép RAG sử dụng nhiều nguồn kiến thức khác nhau được tích hợp sẵn trong các tham số của mô hình và thông tin có trong các đoạn văn ngữ cảnh, giúp nó vượt trội hơn các mô hình tiên tiến khác trong các tác vụ như trả lời câu hỏi.

Việc sử dụng RAG không chỉ là thêm dữ liệu, mà là thay đổi cách mô hình xử lý thông tin. Bảng dưới đây so sánh sự khác biệt cốt lõi:

Đặc điểm	LLM Truyền thống (Pre-trained)	Hệ thống RAG (Retrieval-Augmented)
Nguồn tri thức	Gói gọn trong trọng số mô hình (tại thời điểm training).	Kết hợp trọng số và dữ liệu thực tế bên ngoài.
Cập nhật kiến thức	Phải Fine-tuning lại (tốn kém, chậm).	Chỉ cần cập nhật/thêm file vào Vector DB (tức thời).
Độ chính xác	Dễ bị "ảo tưởng" (Hallucination) khi gặp thông tin lạ.	Giảm thiểu ảo tưởng nhờ dựa trên bằng chứng (Grounding).
Khả năng giải trình	Là một "hộp đen", khó biết tại sao nó trả lời như vậy.	Có khả năng trích dẫn nguồn (Citation) cụ thể.
Chi phí triển khai	Thấp (chỉ cần gọi API).	Cao hơn (cần duy trì hạ tầng lưu trữ và tìm kiếm vector).

Bảng 2.1: Bảng so sánh giữa LLM truyền thống và RAG.

¹Kiến trúc có vai trò trọng yếu trong các ứng dụng dịch thuật máy, với chức năng cốt lõi là biến đổi một chuỗi thông tin đầu vào thành một chuỗi thông tin đầu ra.

Quy trình của RAG thường trải qua 4 giai đoạn chính [3]:

- **Lập chỉ mục (Indexing):** Tài liệu môn học được chia nhỏ (Chunking), chuyển đổi sang không gian vector (Embedding) và lưu vào cơ sở dữ liệu.
- **Truy xuất (Retrieval):** Khi người dùng đặt câu hỏi, hệ thống chuyển câu hỏi thành vector và tìm các đoạn văn bản tương đồng nhất.
- **Làm giàu ngữ cảnh (Augmentation):** Câu hỏi gốc được bao bọc bởi các đoạn văn bản vừa tìm thấy cùng một câu lệnh (Prompt) hướng dẫn.
- **Sinh câu trả lời (Generation):** LLM đọc gói dữ liệu đã được làm giàu để đưa ra phản hồi cuối cùng.

2.1.3. Lý do chọn RAG

Trong đồ án này, RAG được lựa chọn thay vì chỉ Fine-tuning LLM vì các lý do sau:

- **Tính linh hoạt:** Khi tài liệu môn học thay đổi, nhóm chỉ cần cập nhật Vector Database thay vì phải huấn luyện lại toàn bộ mô hình (vốn rất tốn kém tài nguyên GPU).
- **Độ tin cậy:** RAG cho phép hệ thống trích dẫn nguồn (citation), giúp người dùng có thể đối chiếu lại với giáo trình gốc.
- **Kiểm soát phạm vi:** Đảm bảo Chatbot chỉ trả lời dựa trên kiến thức từ các dữ liệu có sẵn, tránh việc mô hình lan man sang các chủ đề không liên quan.

Cũng theo bài báo trên IBM [4], sự khác biệt giữa RAG và Fine-tuning là RAG cho phép LLM truy vấn nguồn dữ liệu bên ngoài, trong khi Fine-tuning huấn luyện LLM trên dữ liệu chuyên biệt theo lĩnh vực. Cả hai đều có cùng mục tiêu chung: làm cho LLM hoạt động tốt hơn trong một lĩnh vực cụ thể.

RAG và Fine-tuning thường được đối lập nhau nhưng có thể được sử dụng song song. Tinh chỉnh giúp mô hình làm quen hơn với miền dự định và các yêu cầu đầu ra, trong khi RAG hỗ trợ mô hình tạo ra các đầu ra chất lượng cao và phù hợp.

2.2 Các kỹ thuật tìm kiếm thông tin

2.2.1. Tìm kiếm từ khóa

Tìm kiếm từ khóa (Keyword Search), hay còn gọi là truy xuất từ vựng (Lexical Retrieval), là phương pháp tìm kiếm dựa trên sự trùng khớp chính xác các ký tự giữa câu truy vấn (Query) và tài liệu (Document).

Trong bối cảnh dữ liệu môn học chuyên ngành (đối tượng nghiên cứu của đồ án), tìm kiếm từ khóa đóng vai trò không thể thay thế khi xử lý:

- Thuật ngữ chuyên môn: Các từ viết tắt, mã học phần, hoặc tên riêng kỹ thuật mà các mô hình ngôn ngữ có thể chưa được học kỹ.
- Độ chính xác tuyệt đối: Khi người dùng muốn tìm chính xác một định nghĩa hoặc một hằng số cụ thể trong giáo trình.

BM25 là một hàm xếp hạng (ranking function) được sử dụng bởi các công cụ tìm kiếm để ước tính độ liên quan của các tài liệu đối với một câu truy vấn nhất định. Nó là sự cải tiến dựa trên nền tảng của mô hình TF-IDF (Term Frequency-Inverse Document Frequency)

Điểm số BM25 của một tài liệu D đối với truy vấn Q (chứa các từ khóa $q_1, q_2, \dots, q_n$), theo một bài báo trên trang Elastic [5], được tính như sau:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Trong đó:

- $f(q_i, D)$: Tần suất xuất hiện của từ khóa q_i trong tài liệu D .
- $|D|$: Độ dài của tài liệu D (số lượng từ).
- avgdl : Độ dài trung bình của tất cả tài liệu trong kho dữ liệu (corpus).
- k_1 và b : Các tham số tự do (thường mặc định $k_1 \in [1.2, 2.0]$ và $b = 0.75$).
- $\text{IDF}(q_i)$: Trọng số nghịch đảo của tài liệu, tính bằng:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

(với N là tổng số tài liệu và $n(q_i)$ là số tài liệu chứa từ q_i).

BM25 giải quyết được hai vấn đề lớn mà TF-IDF gặp phải:

- Sự bão hòa của tần suất từ khóa (Term Frequency Saturation): Ở TF-IDF, nếu một từ xuất hiện càng nhiều, điểm số sẽ tăng tuyến tính vô hạn. Ở BM25, khi tần suất từ khóa đạt đến một ngưỡng nhất định, việc xuất hiện thêm từ đó sẽ không làm tăng đáng kể điểm số (do tham số k_1 điều tiết). Điều này phản ánh đúng thực tế: một đoạn văn lặp lại từ "Machine Learning" 100 lần không nhất thiết phải liên quan gấp 10 lần đoạn văn chứa 10 lần từ đó.
- Chuẩn hóa độ dài tài liệu (Document Length Normalization): Các tài liệu dài

thường có xu hướng chứa nhiều từ khóa hơn một cách ngẫu nhiên. Tham số b trong BM25 giúp "phạt" các tài liệu quá dài và ưu tiên các tài liệu ngắn nhưng súc tích, chứa đúng từ khóa cần tìm.

Mặc dù mạnh mẽ, BM25 có một nhược điểm chí mạng gọi là Vocabulary Mismatch (Bất tương đồng từ vựng). Nếu người dùng hỏi về "cách lưu trữ dữ liệu" nhưng tài liệu lại dùng từ "quản lý cơ sở dữ liệu", BM25 sẽ không thể tìm thấy do không có sự trùng khớp về mặt ký tự, mặc dù hai khái niệm này có sự tương đồng về ngữ nghĩa. Đây chính là lý do cần phải kết hợp với Semantic Search ở các phần tiếp theo.

2.2.2. Tìm kiếm ngữ nghĩa

Tìm kiếm ngữ nghĩa (Semantic Search) là kỹ thuật truy xuất thông tin dựa trên ý nghĩa nội hàm của câu hỏi thay vì chỉ dựa vào các từ khóa xuất hiện trong văn bản.

Trong hệ thống RAG cho tài liệu môn học, kỹ thuật này giải quyết vấn đề Bất tương đồng từ vựng (Vocabulary Mismatch) đã nêu ở trên. Ví dụ: Nếu sinh viên hỏi "Làm sao để bảo mật dữ liệu?", hệ thống tìm kiếm ngữ nghĩa có thể tìm thấy các đoạn văn bản nói về "mã hóa thông tin" hoặc "phân quyền truy cập", dù các từ khóa này không hề xuất hiện trong câu hỏi.

Cốt lõi của tìm kiếm ngữ nghĩa là việc chuyển đổi các thực thể văn bản (từ, câu, hoặc đoạn văn) thành các vector số thực đa chiều trong một không gian gọi là Không gian nhúng (Embedding Space).

- Đặc điểm: Các vector này là "đầy" (dense) vì hầu hết các phần tử đều khác 0, và thường có số chiều cố định (ví dụ: 768 chiều với mô hình BERT-base).
- Nguyên lý: Các đoạn văn bản có ý nghĩa tương đồng sẽ được ánh xạ vào các vị trí nằm gần nhau trong không gian vector.

Để xác định tài liệu nào liên quan nhất đến truy vấn q , hệ thống tính toán khoảng cách hoặc góc giữa vector truy vấn $\mathbf{v}_q$ và vector tài liệu $\mathbf{v}_d$. Công thức phổ biến nhất là Độ tương đồng Cosine (Cosine Similarity):

$$\text{similarity}(\mathbf{v}_q, \mathbf{v}_d) = \cos(\theta) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|}$$

Kết quả nằm trong khoảng $[-1, 1]$, trong đó giá trị gần 1 thể hiện sự tương đồng cao về mặt ngữ nghĩa.

Trong giai đoạn truy xuất (Retrieval), kiến trúc Bi-Encoder (thường dựa trên

Sentence-BERT)[6] là lựa chọn tối ưu về hiệu năng. Tại đây, truy vấn và tài liệu được xử lý độc lập bởi cùng một mô hình Transformer (siamese network) để tạo ra các vector đại diện. Nó cho phép tính toán trước (pre-compute) toàn bộ vector cho kho tài liệu khổng lồ và lưu vào Cơ sở dữ liệu Vector (như FAISS, Milvus). Khi có truy vấn mới, hệ thống chỉ cần tính vector cho câu hỏi và thực hiện tìm kiếm láng giềng gần nhất (ANN search), giúp tốc độ phản hồi đạt mức mili giây (*ms*).

Ưu điểm của dạng tìm kiếm này là hiểu được từ đồng nghĩa, ngữ cảnh và cấu trúc ngôn ngữ phức tạp, ngoài ra còn có khả năng xử lý các câu hỏi diễn đạt theo nhiều cách khác nhau. Tuy nhiên, hạn chế của dạng tìm kiếm này là có thể gặp khó khăn với các từ viết tắt cực kỳ chuyên biệt hoặc các con số chính xác nếu mô hình nhúng (Embedding Model) chưa được huấn luyện trên miền dữ liệu đó.

2.2.3. Tìm kiếm lai

Dù RAG giải quyết được vấn đề kiến thức, nhưng nếu bước Retrieval (Truy xuất) trả về kết quả sai thì Generator cũng sẽ sai (Garbage in, Garbage out). Đó là lý do chúng em đề xuất Hybrid Search và Reranking.

Tìm kiếm lai (Hybrid Search) là phương pháp kết hợp đồng thời hai kỹ thuật: Tìm kiếm từ khóa (Lexical Search - BM25) và Tìm kiếm ngữ nghĩa (Semantic Search - Dense Retrieval).

Trong đề án này, Hybrid Search được đề xuất nhằm khắc phục các "điểm mù" của từng phương pháp riêng lẻ:

- BM25 thường bỏ lỡ các tài liệu có ý nghĩa tương đương nhưng khác từ dùng.
- Semantic Search đôi khi quá tập trung vào ngữ cảnh chung mà bỏ qua các từ khóa quan trọng (như mã môn học, tên riêng của thuật toán, hoặc các thông số kỹ thuật chính xác).

Thách thức lớn nhất của Hybrid Search là làm sao kết hợp điểm số từ hai hệ thống khác nhau (BM25 trả về điểm số thực không giới hạn, trong khi Cosine Similarity trả về giá trị từ -1 đến 1). Phương pháp phổ biến và hiệu quả nhất hiện nay là Reciprocal Rank Fusion (RRF), được giới thiệu bởi Gordon V. Cormack và các cộng sự từ những năm cuối 2010 [7].

RRF (Reciprocal Rank Fusion) không quan tâm đến điểm số tuyệt đối mà chỉ tập trung vào thứ hạng (rank) của tài liệu trong từng danh sách kết quả. Công thức tính điểm RRF cho một tài liệu d là:

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)}$$

Tiêu chí	Tìm kiếm từ khóa	Tìm kiếm ngữ nghĩa	Tìm kiếm lai
Cơ chế	Khớp ký tự chính xác	Khớp ý nghĩa vector	Khớp ý nghĩa vector
Thế mạnh	Thuật ngữ chuyên ngành, mã số, tên riêng	Từ đồng nghĩa, câu hỏi diễn đạt tự nhiên	Toàn diện nhất
Điểm yếu	Lỗi "Vocabulary Mismatch"	Có thể bỏ sót từ khóa chính xác	Cần tài nguyên cho cả 2 hệ thống
Độ chính xác	Trung bình	Cao	Rất cao

Bảng 2.2: Bảng so sánh giữa ba phương pháp tìm kiếm.

Trong đó:

- R : Tập hợp các danh sách kết quả (từ BM25 và Semantic Search).
- $r(d)$: Thứ hạng của tài liệu d trong danh sách r .
- k : Một hằng số làm mượt (smoothing constant), thường được đặt mặc định là 60 để giảm thiểu ảnh hưởng của các tài liệu có thứ hạng rất thấp.

Bảng dưới đây tóm tắt sự khác biệt giữa ba phương pháp để làm rõ lý do tại sao Hybrid Search là lựa chọn tối ưu cho RAG:

Trong thực tế triển khai đồ án với dữ liệu môn học, Hybrid Search mang lại những lợi ích cụ thể:

- Xử lý truy vấn hỗn hợp: Khi sinh viên hỏi "Cấu trúc gói tin TCP", BM25 sẽ bắt đúng từ khóa "TCP", trong khi Semantic Search sẽ hiểu ngữ cảnh về "cấu trúc gói tin".
- Khả năng chịu lỗi: Nếu sinh viên viết sai chính tả nhẹ hoặc dùng thuật ngữ không chuẩn, phần Semantic vẫn có thể cứu vãn kết quả truy xuất.
- Độ ổn định: Giúp hệ thống hoạt động tốt cả với những tài liệu có mật độ từ khóa thấp nhưng thông tin giá trị cao.

2.3 Mô hình xếp hạng lại

Trong các hệ thống truy xuất quy mô lớn, việc tìm kiếm trực tiếp bằng các mô hình phức tạp trên hàng triệu tài liệu là bất khả thi về mặt thời gian. Do đó, quy trình thường được chia thành hai giai đoạn (Two-stage Retrieval):

- Giai đoạn 1 (Retrieval): Sử dụng các mô hình nhanh (như BM25 hoặc Bi-Encoder) để thu hẹp phạm vi từ hàng triệu xuống còn khoảng 50–100 tài liệu

ứng viên.

- Giai đoạn 2 (Reranking): Sử dụng một mô hình có khả năng hiểu sâu về ngữ cảnh để tính toán lại điểm số và xếp hạng lại danh sách ứng viên này.

Mô hình xếp hạng lại đóng vai trò như một "phễu lọc tinh", đảm bảo rằng các thông tin nhiễu bị loại bỏ và chỉ những nội dung có sự tương quan cao nhất mới được đưa vào ngữ cảnh (Context) của LLM.

2.3.1. Kiến trúc Bi-Encoder và Cross-Encoder

Sự khác biệt cốt lõi giữa các phương pháp truy xuất và xếp hạng lại nằm ở cách chúng xử lý mối quan hệ giữa câu truy vấn (Query - Q) và tài liệu (Document - D). Có hai kiến trúc xếp hạng phổ biến là Bi-Encoder (Mô hình mã hóa kép) và Cross-Encoder (Mô hình mã hóa chéo)

Như đã đề cập ở phần 2.2.2, Bi-Encoder xử lý Q và D một cách độc lập thông qua hai nhánh mạng Transformer có chung trọng số (Siamese Network). Câu truy vấn và tài liệu được mã hóa thành hai vector riêng biệt $\mathbf{u}$ và $\mathbf{v}$ trong không gian nhúng. Độ tương đồng được tính nhanh chóng thông qua các phép toán vector đơn giản như Cosine Similarity hoặc Dot Product:

$$Score_{Bi} = \cos(\mathbf{u}, \mathbf{v})$$

Không có sự tương tác giữa các từ trong Q và các từ trong D trong suốt quá trình mã hóa. Điều này làm cho Bi-Encoder có tốc độ cực nhanh nhưng đôi khi bỏ lỡ các mối liên hệ ngữ nghĩa phức tạp.

Khác với Bi-Encoder, Cross-Encoder đưa cả câu truy vấn và tài liệu vào mô hình Transformer cùng một lúc như một cặp đầu vào duy nhất (thường được phân tách bởi ký hiệu [SEP]).

Mô hình cho phép cơ chế Self-Attention hoạt động đồng thời trên tất cả các token của cả Q và D . Điều này có nghĩa là mỗi từ trong câu hỏi có thể trực tiếp "tương tác" với từng từ trong đoạn văn bản để hiểu rõ mối quan hệ logic[8].

Thay vì tạo ra vector nhúng, Cross-Encoder trả về một giá trị thực (thường thông qua một lớp phân loại tuyến tính trên token [CLS]) đại diện cho độ phù hợp giữa chúng:

$$Score_{Cross} = f_{\phi}(Q \oplus D)$$

Để khách quan hóa hai kiến trúc xếp hạng, ta có bảng sau:

Tiêu chí	Tìm kiếm từ khóa	Tìm kiếm ngữ nghĩa
Độ tương tác ngữ nghĩa	Thấp (Chỉ tương tác ở lớp cuối cùng).	Rất cao (Tương tác ở mọi lớp Attention).
Độ chính xác	Khá.	Rất cao.
Tốc độ xử lý Khả năng mở rộng	Cực nhanh (Hỗ trợ lập chỉ mục trước). Có thể tìm kiếm trên hàng tỷ tài liệu.	Chậm (Phải tính toán thời gian thực). Chỉ phù hợp xếp hạng lại một nhóm nhỏ (<100).
Ứng dụng trong đồ án	Dùng để lọc thô các đoạn giáo trình.	Dùng để chọn ra 3-5 đoạn văn chuẩn nhất.

Bảng 2.3: Bảng so sánh giữa ba phương pháp tìm kiếm.

Mô hình Cross-Encoder đạt hiệu suất cao hơn Bi-Encoder, tuy nhiên, chúng không mở rộng tốt đối với các tập dữ liệu lớn. Vì thế, trong nhiều trường hợp, việc kết hợp Cross-Encoder và Bi-Encoder có thể mang lại hiệu quả, ví dụ như trong các kịch bản Truy xuất thông tin / Tìm kiếm ngữ nghĩa[9][6].

Ví dụ: đầu tiên, bạn sử dụng một Bi-Encoder hiệu quả để truy xuất ví dụ như 100 câu có độ tương đồng cao nhất cho một truy vấn. Sau đó, bạn sử dụng một Cross-Encoder để xếp hạng lại 100 kết quả này bằng cách tính điểm cho mỗi sự kết hợp (truy vấn, kết quả).

2.3.2. Mô hình PhoRanker và ứng dụng cho tiếng Việt

Mặc dù các mô hình đa ngôn ngữ (Multilingual Models) như mBERT hay XLM-RoBERTa đã đạt được những thành tựu nhất định, việc áp dụng chúng vào bài toán xếp hạng văn bản tiếng Việt vẫn gặp phải nhiều rào cản. PhoRanker (phát triển bởi nhóm nghiên cứu itdainb) ra đời nhằm giải quyết các vấn đề cốt lõi sau:

- Sự hạn chế của mô hình đa ngôn ngữ: Các mô hình như mBERT thường sử dụng bộ từ điển (vocabulary) chung cho hàng trăm ngôn ngữ, dẫn đến việc mã hóa tiếng Việt không tối ưu (hiện tượng vỡ chữ/subword không mang ý nghĩa ngữ nghĩa hoàn chỉnh).

- Khoảng cách về ngữ cảnh địa phương: Tiếng Việt có cấu trúc ngữ pháp đặc thù và hiện tượng đa nghĩa (ambiguity) cao. Các mô hình Cross-Encoder phổ biến trên thế giới chủ yếu được huấn luyện trên dữ liệu tiếng Anh (như MS MARCO), dẫn đến hiệu năng giảm sút khi xử lý các truy vấn tiếng Việt phức tạp.
- Sự thiếu hụt công cụ Reranking chuyên biệt: Trước PhoRanker, cộng đồng nghiên cứu tại Việt Nam chủ yếu tập trung vào các mô hình Embedding (Bi-Encoder). Sự thiếu vắng một Cross-Encoder mạnh mẽ khiến giai đoạn xếp hạng lại trong quy trình RAG trở nên yếu thế, làm giảm độ chính xác của câu trả lời cuối cùng.

PhoRanker được xây dựng dựa trên kiến trúc Cross-Encoder, sử dụng mô hình nền tảng là PhoBERT (*RoBERTa_{base}* phiên bản cho tiếng Việt)[10].

Kế thừa từ PhoBERT, PhoRanker sử dụng công nghệ tách từ (Word Segmentation) giúp xử lý các từ ghép trong tiếng Việt hiệu quả hơn so với cách tách subword thông thường.

Tương tự như kiến trúc Cross-Encoder tổng quát, PhoRanker nhận đầu vào là cặp [CLS] Query [SEP] Document [SEP]. Tại lớp đầu ra, mô hình sử dụng hàm kích hoạt để dự đoán điểm số tương quan (Relevancy Score) giữa câu hỏi và đoạn văn bản. [11]

Để đạt được hiệu năng vượt trội, PhoRanker được huấn luyện thông qua quá trình:

- Kiến thức chuyển đổi (Transfer Learning): Tận dụng tri thức từ các mô hình Cross-Encoder đã thành công trên thế giới, sau đó thực hiện Fine-tuning trên các bộ dữ liệu tiếng Việt quy mô lớn.
- Dữ liệu huấn luyện chất lượng cao: Sử dụng các bộ dữ liệu như ViMS (Vietnamese Multiple Document Summarization) hoặc các bộ QA chuyên biệt để mô hình học được cách phân biệt giữa các đoạn văn bản có vẻ giống nhau nhưng chỉ một đoạn chứa câu trả lời đúng (Hard Negatives).

Mô hình này có thể xử lý 15 tài liệu mỗi giây với fp16 trên GPU A100. PhoRanker còn mang lại hiệu năng ấn tượng, vượt trội hơn các mô hình khác trong bộ dữ liệu MS MARCO Passage Reranking - Vi - Dev. Với điểm số 0.6625 trong NDCG@3 và 0.6458 trong MRR@3[12]. Bằng cách tận dụng kiến trúc Cross-Encoder, PhoRanker có thể xếp hạng văn bản một cách hiệu quả với độ chính xác cao.

Trong đề án này, PhoRanker đóng vai trò là "chốt chặn" cuối cùng của quy trình tìm kiếm:

- Lọc nhiễu: Loại bỏ các đoạn văn bản mà Hybrid Search đã lấy về nhưng thực tế không liên quan chặt chẽ đến câu hỏi.
- Tối ưu hóa cửa sổ ngữ cảnh (Context Window): Vì các LLM (như GPT-4 hay Llama 3) có giới hạn về số lượng token và dễ bị nhiễu bởi thông tin dư thừa, PhoRanker giúp chọn ra top 3-5 đoạn văn bản tinh túy nhất, giúp LLM sinh câu trả lời chính xác và ngắn gọn hơn.

2.4 Kỹ thuật mở rộng truy vấn

Kỹ thuật mở rộng truy vấn (Query Expansion) là quá trình bổ sung các từ ngữ, cụm từ liên quan hoặc tạo ra các biến thể mới từ câu truy vấn gốc của người dùng trước khi đưa vào hệ thống tìm kiếm.

Trong hệ thống RAG, kỹ thuật này giải quyết hai vấn đề lớn:

- Sự ngắn gọn (Query Sparsity): Câu hỏi của người dùng thường quá ngắn, thiếu các từ khóa cần thiết để thuật toán BM25 hay Vector Search hoạt động hiệu quả.
- Sự lệch pha về phong cách (Style Mismatch): Người dùng thường đặt câu hỏi theo dạng "ngôn ngữ nói", trong khi tài liệu môn học lại viết theo dạng "ngôn ngữ học thuật".

2.4.1. Kỹ thuật Sinh truy vấn đa dạng (Multi-Query/Generated Queries)

Đây là kỹ thuật sử dụng một mô hình ngôn ngữ (thường là một LLM nhẹ hoặc Prompt-based) để tạo ra nhiều phiên bản khác nhau của cùng một câu hỏi từ các góc độ khác nhau. Kỹ thuật này dựa trên giả định rằng: Một câu hỏi đơn lẻ có thể không đủ để bao quát toàn bộ đặc trưng vector của tài liệu mục tiêu.

Với câu hỏi Q ban đầu, LLM sẽ sinh ra các câu hỏi $\{Q_1, Q_2, \dots, Q_n\}$. Hệ thống sẽ thực hiện truy xuất cho tất cả các câu hỏi này.

Cơ chế hoạt động của kỹ thuật này gồm ba bước:

1. Phân tích (Query Rewriting): LLM nhận câu hỏi gốc Q và được yêu cầu sinh ra n biến thể (ví dụ $n = 3$) có cùng ý nghĩa nhưng khác cách diễn đạt (ví dụ: đổi từ chủ động sang bị động, dùng từ đồng nghĩa chuyên ngành).
2. Truy xuất song song: Mỗi truy vấn Q_i sẽ được chuyển thành vector và tìm kiếm độc lập trong Vector Database.
3. Hợp nhất kết quả (Union): Hệ thống lấy tập hợp các tài liệu từ tất cả các kết quả trả về, loại bỏ trùng lặp.

Kỹ thuật này làm giúp tăng khả năng bao phủ (Recall). Nếu câu hỏi gốc không tìm thấy tài liệu, các biến thể (dùng từ đồng nghĩa hoặc cấu trúc khác) có thể chạm tới đúng đoạn văn bản cần tìm. Nó đảm bảo rằng dù người dùng dùng từ ngữ bình dân hay học thuật, hệ thống vẫn tìm thấy tài liệu.

2.4.2. Kỹ thuật Nhúng tài liệu giả định (Hypothetical Document Embeddings - HyDE)

HyDE là một kỹ thuật đột phá được đề xuất bởi Gao et al. (2022) [13]. Thay vì dùng câu hỏi để tìm tài liệu, HyDE sử dụng câu hỏi để sinh ra một "tài liệu giả định". HyDE giải quyết vấn đề "Sự bất đối xứng giữa Câu hỏi và Câu trả lời" trong không gian vector. Trong thực tế, vector của một "Câu hỏi" và vector của một "Đoạn văn chứa câu trả lời" thường không nằm gần nhau bằng hai vector của hai "Đoạn văn" cùng chủ đề.

Quy trình được thực hiện qua ba bước sau:

1. Người dùng đặt câu hỏi Q .
2. LLM sinh ra một câu trả lời giả định D_{hypo} (mô hình không cần quan tâm tính đúng sai của câu trả lời này, chỉ cần đúng định dạng tài liệu).
3. Hệ thống dùng vector của D_{hypo} để tìm các tài liệu thực D_{real} trong cơ sở dữ liệu.

Trong không gian vector, một "câu trả lời giả định" thường nằm gần các "câu trả lời thật" hơn là một "câu hỏi". Điều này giúp thu hẹp khoảng cách ngữ nghĩa đáng kể. Ngay cả khi D_{hypo} chứa thông tin sai (ảo tưởng), nó vẫn chứa các từ khóa và ngữ cảnh đặc trưng của chủ đề đó. Việc so sánh "Văn bản với Văn bản" luôn mang lại độ chính xác cao hơn "Câu hỏi với Văn bản".

2.4.3. Mối quan hệ giữa Multi-Query và HyDE)

1. Mặc dù là hai kỹ thuật độc lập, chúng có mối liên hệ mật thiết và có thể phối hợp trong một hệ thống RAG nâng cao:
2. Tính bổ trợ về không gian (Space Complementarity):
 - Multi-Query mở rộng bề ngang của vùng tìm kiếm (quét nhiều vị trí trong không gian vector để tránh bỏ sót).
 - HyDE thực hiện một "cú nhảy" về mặt vị trí (chuyển từ vùng chứa các câu hỏi sang vùng chứa các tài liệu trả lời).
3. Sự kết hợp (Hybrid Expansion): Một hệ thống mạnh mẽ có thể sử dụng LLM

để sinh ra nhiều câu trả lời giả định khác nhau cho cùng một câu hỏi. Điều này tạo ra một "vùng hội tụ" giúp xác định cực kỳ chính xác tài liệu mục tiêu.

4. Sự khác biệt về chi phí: Multi-Query tốn nhiều tài nguyên hơn ở giai đoạn tìm kiếm (vì phải Search n lần), trong khi HyDE tốn tài nguyên ở giai đoạn LLM sinh văn bản (vì phải viết một đoạn văn dài thay vì các câu hỏi ngắn).

Việc áp dụng Query Expansion trong đồ án mang lại lợi thế đặc biệt khi đối mặt với dữ liệu chuyên ngành:

- Giải quyết từ viết tắt: Nếu sinh viên hỏi về "CSDL", cơ chế mở rộng có thể tự động thêm từ "Cơ sở dữ liệu", giúp BM25 bắt đúng các đoạn văn bản học thuật.
- Làm rõ ngữ cảnh: Biến những câu hỏi cụt ngắn thành những truy vấn đầy đủ thông tin, giúp Reranker (PhoRanker) ở bước sau có đủ dữ liệu để xếp hạng chính xác.

2.5 Các chỉ số đánh giá hệ thống

Việc đánh giá một hệ thống RAG phức tạp hơn so với các bài toán ML thông thường vì nó đòi hỏi sự kiểm định ở cả hai giai đoạn: Truy xuất (Retrieval) và Sinh câu trả lời (Generation). Một kết quả sai có thể đến từ việc tìm kiếm không đúng tài liệu hoặc mô hình ngôn ngữ hiểu sai ngữ cảnh được cung cấp.

Mục tiêu của giai đoạn này là tìm được các đoạn văn bản chứa câu trả lời và xếp chúng ở vị trí cao nhất có thể trong danh sách kết quả.

2.5.1. Chỉ số đánh giá giai đoạn Truy xuất (Retrieval Metrics)

2.5.1a. Recall@k (Độ bao phủ tại k)

Recall@k đo lường tỷ lệ các câu hỏi mà tài liệu đúng (ground truth) xuất hiện trong top k kết quả trả về. Trong RAG, Recall@k cực kỳ quan trọng ở giai đoạn Retrieval thô. Nếu tài liệu đúng không nằm trong top k , giai đoạn Reranking và Generation sau đó sẽ không có dữ liệu để xử lý.

Công thức cơ bản của chỉ số được tính như sau:

$$Recall@k = \frac{\text{Số lượng tài liệu liên quan nằm trong top } k}{\text{Tổng số tài liệu liên quan trong tập dữ liệu}}$$

2.5.1b. MRR (Mean Reciprocal Rank - Xếp hạng nghịch đảo trung bình)

MRR đánh giá khả năng của hệ thống trong việc đưa tài liệu chính xác lên vị trí đầu tiên. Chỉ số này đặc biệt quan trọng khi đánh giá mô hình PhoRanker. Nếu tài liệu đúng nằm ở vị trí thứ 1, điểm là 1; nếu nằm ở vị trí thứ 2, điểm là 0.5.

Công thức của MRR được tính:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

, trong đó $|Q|$ là tổng số câu hỏi, $rank_i$ là vị trí của tài liệu đúng đầu tiên tìm được cho câu hỏi i .

2.5.1c. NDCG (Normalized Discounted Cumulative Gain)

NDCG đánh giá chất lượng xếp hạng dựa trên độ liên quan của tài liệu, trong đó các tài liệu nằm ở vị trí càng cao thì trọng số càng lớn. Khác với MRR chỉ quan tâm đến kết quả đúng đầu tiên, NDCG xem xét toàn bộ thứ tự của danh sách. Điều này phù hợp để đánh giá Hybrid Search khi kết hợp nhiều nguồn kết quả.

2.5.2. Chỉ số đánh giá giai đoạn Sinh câu trả lời (Generation Metrics)

Để đánh giá chất lượng câu trả lời từ LLM, thay vì dùng các chỉ số truyền thống như BLEU hay ROUGE (vốn chỉ khớp từ vựng), đề án sử dụng khung đánh giá RAGAS (Retrieval-Augmented Generation Assessment). Chỉ số này phụ thuộc vào hai cơ sở: tính trung thực của câu trả lời và sự liên quan tới câu trả lời[14].

2.5.2a. Tính trung thực (Faithfulness)

Chỉ số này đo lường mức độ mà câu trả lời được sinh ra hoàn toàn dựa trên ngữ cảnh (Context) được cung cấp, thay vì từ kiến thức sẵn có của LLM. Dựa vào chỉ số này, có thể kiểm soát hiện tượng "ảo tưởng" (Hallucination). Một điểm số cao cho thấy chatbot không "tự chế" ra thông tin ngoài tài liệu môn học.

2.5.2b. Sự liên quan của câu trả lời (Answer Relevance)

Đo lường mức độ mà câu trả lời trực tiếp giải quyết vấn đề được nêu trong câu hỏi của người dùng, từ đó đảm bảo câu trả lời không bị lan man hoặc trả lời đúng kiến thức nhưng sai trọng tâm câu hỏi.

Dựa trên những chỉ số đã nêu ở trên, ta có bảng tổng kết:

Giai đoạn	Chỉ số	Mục tiêu đo lường
Retrieval	Recall@k	Đảm bảo tài liệu đúng không bị bỏ sót.
Retrieval	MRR	Đảm bảo tài liệu đúng nằm ở top đầu (đánh giá Reranker).
Retrieval	NDCG	Đánh giá tổng thể chất lượng xếp hạng.
Generation	Faithfulness	Đảm bảo câu trả lời căn cứ vào tài liệu (tránh ảo tưởng).
Generation	Answer Relevance	Đảm bảo câu trả lời đúng trọng tâm câu hỏi.

Bảng 2.4: Bảng tổng kết các chỉ số.

Chương 3: Phương pháp đề xuất

3.1 Kiến trúc hệ thống tổng quát

Kiến trúc tổng thể vận hành thông qua hai luồng xử lý song song và tương hỗ: (1) Luồng đánh chỉ mục dữ liệu ngoại tuyến (Offline Indexing Pipeline) và (2) Luồng truy vấn thời gian thực (Real-time Inference Pipeline).

3.1.1. Luồng đánh chỉ mục dữ liệu (Offline Indexing Pipeline)

Luồng đánh chỉ mục đóng vai trò nền tảng trong kiến trúc RAG, chịu trách nhiệm chuyển đổi kho tri thức thô (unstructured knowledge base) thành cấu trúc dữ liệu vector có khả năng truy hồi hiệu quả. Quy trình này được thực hiện theo cơ chế bất đồng bộ (asynchronous) để đảm bảo hiệu năng hệ thống khi xử lý lượng dữ liệu lớn. Các bước xử lý cụ thể như sau:

a. Thu thập và Tiền xử lý dữ liệu (Data Ingestion và Preprocessing)

Hệ thống được thiết kế để tiếp nhận đa dạng định dạng tài liệu đầu vào. Tại bước này, module xử lý tập trung vào hai nhiệm vụ chính:

- Trích xuất văn bản (Text Extraction): Sử dụng các thư viện chuyên dụng để bóc tách nội dung văn bản khỏi định dạng trình bày.
- Kiểm tra tính toàn vẹn và Loại bỏ trùng lặp: Trước khi xử lý sâu, mỗi tài liệu được tạo một mã băm định danh duy nhất (SHA-256 hash). Hệ thống sẽ so khớp mã này với cơ sở dữ liệu hiện có để ngăn chặn việc đánh chỉ mục trùng lặp, giúp tối ưu hóa tài nguyên lưu trữ và tính toán.

b. Chiến lược phân đoạn văn bản (Text Chunking Strategy)

Do giới hạn về độ dài ngữ cảnh (context window) của mô hình nhúng và mô hình ngôn ngữ, dữ liệu không thể được xử lý nguyên khối. Hệ thống áp dụng chiến lược phân đoạn cửa sổ trượt (sliding window) với các tham số cấu hình tối ưu:

- Kích thước đoạn (Chunk size): Được thiết lập ở mức 400-500 tokens. Kích thước này được lựa chọn dựa trên thực nghiệm để đảm bảo mỗi đoạn chứa đủ thông tin ngữ nghĩa trọn vẹn của một ý tưởng hoặc đoạn văn.
- Độ chồng lấp (Overlap): Thiết lập mức 50 tokens giữa các đoạn liên tiếp. Cơ chế này đóng vai trò quan trọng trong việc bảo toàn ngữ nghĩa tại các điểm cắt, đảm bảo thông tin quan trọng nằm ở biên giới giữa hai đoạn không bị đứt gãy hoặc mất mát ngữ cảnh.

c. Mã hóa Vector (Vector Embedding Generation)

Đây là bước chuyển đổi dữ liệu từ không gian ngôn ngữ tự nhiên sang không gian vector nhiều chiều (high-dimensional vector space).

- Hệ thống sử dụng mô hình `dangvantuan/vietnamese-document-embedding` để mã hóa từng đoạn văn bản thành một vector đặc trưng có kích thước 768 chiều. Đây là mô hình đơn ngữ (monolingual) được huấn luyện chuyên sâu trên ngữ liệu tiếng Việt khổng lồ, cho khả năng biểu diễn ngữ nghĩa vượt trội so với các mô hình đa ngôn ngữ (multilingual) thông thường.
- Mô hình chuyển đổi từng đoạn văn bản thành vector đặc trưng có kích thước 768 chiều (kích thước tiêu chuẩn của kiến trúc BERT-base), giúp nắm bắt sâu sắc các sắc thái ngữ nghĩa đặc trưng của tiếng Việt.

d. Lưu trữ và Quản lý chỉ mục (Vector Storage & Indexing)

Kết quả mã hóa được lưu trữ trong Qdrant với cấu trúc:

- Vector: Biểu diễn toán học 768 chiều của đoạn văn bản.
- Payload: Chứa nội dung gốc và metadata phong phú để phục vụ truy xuất nguồn gốc.
- Độ đo khoảng cách sử dụng là Cosine Similarity.

3.1.2. Luồng truy vấn và sinh câu trả lời (Real-time Inference Pipeline)

Luồng truy vấn thời gian thực được thiết kế theo mô hình "Retrieve-then-Generate", tích hợp mô hình Reranker đã được tinh chỉnh (fine-tuned) để tối ưu hóa độ chính xác cho miền dữ liệu cụ thể.

a. Phân tích và Mã hóa truy vấn (Query Processing & Embedding)

Câu hỏi người dùng được chuẩn hóa và đưa qua cùng mô hình embedding `dangvantuan/vietnamese-document-embedding` để tạo ra vector truy vấn. Việc sử dụng cùng một mô hình với giai đoạn đánh chỉ mục đảm bảo tính đồng nhất trong không gian vector (vector space alignment).

b. Cơ chế Tìm kiếm Lai (Hybrid Search Mechanism)

Hệ thống kết hợp hai phương pháp tìm kiếm để đảm bảo độ bao phủ (Recall):

- Dense Retrieval (Vector Search): Tìm kiếm dựa trên ngữ nghĩa sử dụng vector 768 chiều.
- Sparse Retrieval (Keyword Search): Tìm kiếm dựa trên từ khóa (BM25) để bắt các thuật ngữ chuyên ngành chính xác.

- Kết quả của hai luồng này được hợp nhất và lấy ra tập ứng viên rộng ($K_{init} \approx 20 - 50$ documents) để chuyển sang bước tái xếp hạng.

c. Tái xếp hạng ngữ cảnh (Semantic Reranking with Fine-tuned Model)

Đây là thành phần cốt lõi tạo nên độ chính xác của hệ thống.

- Mô hình cơ sở: Sử dụng `itdaib/PhoRanker`, một mô hình Cross-Encoder dựa trên kiến trúc RoBERTa, được pre-train mạnh mẽ cho tác vụ xếp hạng văn bản tiếng Việt.
- Tinh chỉnh (Fine-tuning): Điểm khác biệt của hệ thống là mô hình PhoRanker này không sử dụng nguyên bản mà đã được huấn luyện tinh chỉnh (fine-tuned) trên bộ dữ liệu chuyên ngành. Việc fine-tuning giúp mô hình "hiểu" sâu hơn về mối quan hệ giữa câu hỏi và văn bản trong ngữ cảnh cụ thể của bài toán, vượt qua hiệu suất của mô hình gốc (base model).
- Quy trình: Mô hình nhận đầu vào là cặp [Câu hỏi, Đoạn văn], tính toán điểm tương quan (relevance score) và lọc ra top_k đoạn văn bản xuất sắc nhất làm ngữ cảnh.

d. Tổng hợp ngữ cảnh và Sinh câu trả lời (Context Assembly & Generation)

- Prompt Engineering: Ngữ cảnh sau khi được rerank bởi mô hình fine-tuned sẽ được ghép với câu hỏi và đưa vào mô hình Google Gemini.
- Generation: LLM tổng hợp thông tin, sinh câu trả lời tự nhiên và trích dẫn nguồn dựa trên metadata của các đoạn văn bản đã chọn.

3.2 Quy trình xử lý dữ liệu

Trong kiến trúc RAG, chất lượng của dữ liệu đầu vào (Input Data Quality) quyết định trực tiếp đến hiệu suất của mô hình truy hồi. Do đó, quy trình xử lý dữ liệu (Data Pipeline) được thiết kế như một dây chuyền tự động hóa gồm nhiều tầng lọc, nhằm chuyển đổi các tài liệu thô đa dạng định dạng thành các đơn vị thông tin tiêu chuẩn hóa, sạch nhiễu và giàu ngữ nghĩa.

3.2.1. Tiền xử lý và làm sạch dữ liệu

Để đảm bảo không gian vector của tài liệu (Document) và truy vấn (Query) được đồng nhất tối đa, hệ thống áp dụng hai quy trình xử lý riêng biệt nhưng tương thích về mặt chuẩn hóa.

a. Chuẩn hóa Unicode (Unicode Normalization Strategy)

Thay vì sử dụng chuẩn NFC thông thường, hệ thống áp dụng chuẩn NFKC (Normalization Form Compatibility Composition) thông qua thư viện `unicodedata`.

- Cơ chế: NFKC không chỉ đưa các ký tự tổ hợp về dạng dựng sẵn (như NFC) mà còn chuẩn hóa các ký tự "tương thích" về dạng chuẩn cơ bản.
- Lý do lựa chọn: Tài liệu đầu vào thường chứa các ký tự đặc biệt do lỗi định dạng hoặc trích xuất từ PDF. NFKC giúp loại bỏ sự nhập nhằng này triệt để hơn, giúp mô hình embedding "hiểu" từ ngữ chính xác hơn.

b. Làm sạch nhiễu và Xử lý lỗi OCR (Artifact Cleaning)

Đối với văn bản trích xuất từ tài liệu (Document Text), hệ thống áp dụng các bộ lọc nghiêm ngặt để loại bỏ nhiễu:

- Loại bỏ ký tự điều khiển: Sử dụng bộ lọc Category 'C' của Unicode để loại bỏ toàn bộ các ký tự điều khiển không in được (zero-width spaces, control codes), chỉ giữ lại các ký tự định dạng cấu trúc thiết yếu là xuống dòng, về đầu dòng và tab.
- Khử nhiễu OCR đặc thù (Vietnamese OCR Cleaning): Áp dụng kỹ thuật "Danh sách cho phép" (Allowlist Regex). Hệ thống sử dụng biểu thức chính quy (Regex) để chỉ giữ lại các ký tự chữ cái, số, dấu câu cơ bản và toàn bộ bộ ký tự tiếng Việt có dấu (á, à, ả, ã, ạ...). Mọi ký tự lạ (emoji, icon, rác mã hóa) nằm ngoài danh sách này sẽ bị thay thế bằng khoảng trắng. Điều này đặc biệt hiệu quả để xử lý các tài liệu scan chất lượng thấp.

c. Chuẩn hóa cấu trúc văn bản (Structural Normalization)

- Xử lý dấu câu: Các dấu câu lặp lại liên tiếp do lỗi đánh máy (ví dụ: !!, ..., ??) được rút gọn về một ký tự duy nhất (!, ., ?) để tránh gây nhiễu cho mô hình phân tích cú pháp.
- Chuẩn hóa dấu gạch ngang: Thống nhất các biến thể của dấu gạch ngang (En-dash –, Em-dash —) về một dạng hyphen tiêu chuẩn (-).
- Xử lý khoảng trắng: Loại bỏ toàn bộ khoảng trắng thừa (extra whitespace) ở đầu, cuối và giữa các từ.

d. Chiến lược xử lý truy vấn người dùng (Query Preprocessing Strategy)

Khác với quy trình xử lý tài liệu (nặng về làm sạch), quy trình xử lý truy vấn được thiết kế "nhẹ nhàng" (lighter) hơn nhằm bảo toàn ý định (intent) của người dùng:

- Vẫn áp dụng chuẩn hóa NFKC và xử lý khoảng trắng để đồng bộ với dữ liệu trong Vector DB.
-

3.2.2. Chiến lược phân đoạn văn bản (Chunking Strategy)

Khác với các phương pháp phân đoạn cắt lớp cố định (fixed-size splitting) thông thường, hệ thống áp dụng thuật toán Semantic Chunking hướng ngữ cảnh. Thuật toán này được thiết kế để đảm bảo tính toàn vẹn của thông tin ở ba cấp độ: câu văn, đoạn văn và cấu trúc phân cấp của tài liệu.

a. Bảo toàn cấu trúc phân cấp tài liệu (Context-Aware Hierarchy)

Trước khi thực hiện phân đoạn, hệ thống xây dựng bản đồ cấu trúc tài liệu

- Cơ chế: Hệ thống duyệt qua toàn bộ tài liệu để xác định các tiêu đề chương, mục (Section Headers) và cấp độ của chúng (Heading Levels).
- Ứng dụng: Mỗi chunk được sinh ra không chỉ chứa văn bản thuần túy mà còn kế thừa toàn bộ đường dẫn ngữ cảnh. Ví dụ: Một đoạn văn bản thuộc Chương 3 > Mục 3.2 sẽ được gắn metadata này. Điều này giúp mô hình LLM hiểu rõ ngữ cảnh rộng hơn của đoạn văn bản khi truy hồi.

b. Phân tách câu dựa trên mô hình ngôn ngữ (Linguistic Sentence Splitting)

Thay vì sử dụng biểu thức chính quy (Regex) đơn giản dễ gây lỗi với các từ viết tắt (như "Tp.", "Mr."), hệ thống tích hợp thư viện NLTK (Natural Language Toolkit) với bộ tokenizer dành riêng cho tiếng Việt.

- Quy trình: Văn bản thô được tách thành danh sách các câu hoàn chỉnh
- Mục đích: Đảm bảo đơn vị nhỏ nhất của một chunk là một câu văn trọn vẹn, tuyệt đối không cắt ngang giữa câu gây mất ngữ nghĩa.

c. Thuật toán gom nhóm động (Dynamic Accumulation Algorithm)

Hệ thống sử dụng cơ chế "cửa sổ trượt mềm" (soft sliding window) với hai tham số ngưỡng:

- Ngưỡng mục tiêu (Target Tokens - Default: 400): Là kích thước lý tưởng mà hệ thống cố gắng đạt được. Hệ thống sẽ liên tục gom nhóm các câu liên tiếp vào một chunk cho đến khi độ dài vượt qua ngưỡng này.
- Ngưỡng giới hạn cứng (Max Tokens - Default: 512): Là giới hạn vật lý tuyệt đối. Trước khi thêm một câu mới vào chunk, hệ thống kiểm tra: $\text{if current_tokens} + \text{sentence_tokens} \leq \text{max_tokens}$.
 - Nếu thêm câu mới làm vượt quá 512 tokens, hệ thống buộc phải ngắt chunk hiện tại ngay lập tức, dù chưa đạt ngưỡng mục tiêu, để đảm bảo an toàn cho mô hình Embedding.

d. Cơ chế chồng lấp dựa trên câu (Sentence-based Overlap)

- Logic: Thay vì chồng lấp dựa trên số lượng tokens (có thể cắt dở dang một từ), hệ thống thực hiện chồng lấp dựa trên câu.
- Thực thi: Khi một chunk kết thúc, hệ thống sẽ lấy N câu cuối cùng của chunk đó (sao cho tổng độ dài các câu này $\leq \text{overlap_tokens}$, mặc định 50) để làm phần mở đầu cho chunk tiếp theo. Điều này đảm bảo sự liên kết mượt mà và logic giữa các đoạn.

e. Xử lý ngoại lệ (Handling Edge Cases)

Đối với các trường hợp đặc biệt như một câu văn quá dài (vượt quá max_tokens), hệ thống sẽ cố gắng phân tách câu dài đó dựa trên các dấu phẩy (,) hoặc chấm phẩy (;) để chia nhỏ thành các tiểu đoạn (sub-chunks), đảm bảo không có dữ liệu nào bị loại bỏ do quá kích thước.

3.3 Module truy vấn nâng cao

Trong các hệ thống RAG cơ bản, việc truy hồi chỉ dựa trên sự tương đồng vector của câu hỏi gốc thường gặp hạn chế do tính chất ngắn gọn và thiếu ngữ cảnh của truy vấn người dùng. Module truy vấn nâng cao được thiết kế để khắc phục nhược điểm này thông qua hai kỹ thuật chủ đạo: Mở rộng truy vấn (Query Expansion) và Tìm kiếm lai (Hybrid Search).

3.3.1. Cài đặt Query Expansion: Kỹ thuật HyDE (Hypothetical Document Embeddings)

Để giải quyết vấn đề "lệch pha ngữ nghĩa" (semantic mismatch) giữa câu hỏi ngắn gọn của người dùng và văn phong hàn lâm của tài liệu, hệ thống triển khai module QueryExpander sử dụng sức mạnh của mô hình ngôn ngữ lớn (LLM). Thay vì chỉ truy hồi dựa trên một câu hỏi duy nhất, hệ thống sinh ra nhiều biến thể truy vấn khác nhau để quét dữ liệu đa chiều hơn.

a. Kiến trúc đa chiến lược (Multi-strategy Approach)

Hệ thống áp dụng đồng thời ba chiến lược mở rộng để tối ưu hóa khả năng bao phủ (Recall):

- Chiến lược Diễn giải lại (Paraphrasing Strategy):
 - Mục tiêu: Khắc phục sự cứng nhắc trong cách đặt câu hỏi. Người dùng có thể hỏi theo phong cách suồng sã (Casual), nhưng tài liệu lại viết theo phong cách trang trọng (Formal) hoặc kỹ thuật (Technical).
 - Thực thi: Sử dụng LLM để viết lại câu hỏi gốc dưới nhiều văn phong khác

nhau.

- Ví dụ: "Tại sao cây cần ánh sáng?" → "Cơ chế quang hợp của thực vật" (Technical) hoặc "Vai trò của năng lượng mặt trời đối với sự phát triển thực vật" (Formal).
- Chiến lược Phân rã truy vấn (Query Decomposition):
 - Mục tiêu: Xử lý các câu hỏi phức hợp (Complex Queries) mà một đoạn văn bản đơn lẻ không thể trả lời hết.
 - Thực thi: Phân tách câu hỏi "kép" thành các câu hỏi đơn "nguyên tử" để tìm kiếm riêng biệt.
 - Ví dụ: "Tác động môi trường của rác thải nhựa và cách giảm thiểu?" được tách thành hai luồng tìm kiếm: (1) "Tác động môi trường của rác thải nhựa" và (2) "Các phương pháp giảm thiểu rác thải nhựa".
- Chiến lược HyDE cải tiến (Modified Hypothetical Document Embeddings):
 - Mục tiêu: Tìm kiếm dựa trên nội dung câu trả lời kỳ vọng thay vì chỉ dựa trên từ khóa câu hỏi.
 - Thực thi: Hàm `_generate_hypothetical_answers` hoạt động theo quy trình hai bước:
 1. Yêu cầu LLM sinh ra một đoạn văn bản giả định (hypothetical excerpt) có khả năng trả lời câu hỏi.
 2. Trích xuất các từ khóa hoặc cụm từ tìm kiếm (search terms) đắt giá từ đoạn văn giả định đó để làm truy vấn mở rộng.
 - Ưu điểm: Phương pháp này giúp hệ thống "tưởng tượng" ra đích đến trước khi bắt đầu tìm đường, giúp bắt được các tài liệu có từ vựng chuyên ngành mà câu hỏi gốc không hề nhắc tới.

b. Quy trình xử lý và Tối ưu hóa

- Mô hình nền tảng: Sử dụng Gemini 2.5 Flash. Đây là mô hình được lựa chọn tối ưu cho tác vụ này nhờ độ trễ thấp và chi phí hiệu quả, đảm bảo trải nghiệm người dùng thời gian thực.
- Cơ chế gộp và lọc trùng (Aggregation & Deduplication): Kết quả từ các chiến lược trên được gộp vào một danh sách chung. Hệ thống thực hiện chuẩn hóa (lowercase, strip) và loại bỏ các biến thể trùng lặp để tránh lãng phí tài nguyên tính toán ở bước Vector Search tiếp theo.
- Kiểm soát số lượng: Số lượng biến thể được giới hạn bởi tham số `max_variants`

(mặc định là 3 biến thể bổ sung cho câu hỏi gốc) để cân bằng giữa độ chính xác và hiệu năng hệ thống.

c. Prompt Engineering

Các prompt gửi đến LLM được thiết kế tối giản và định hướng rõ ràng đầu ra (Output-oriented Prompts) để dễ dàng phân tích (parse). Ví dụ, prompt cho Paraphrasing yêu cầu "Only output the paraphrases, one per line" (Chỉ xuất các câu diễn giải, mỗi câu một dòng), giúp loại bỏ các lời dẫn dắt thừa thãi của LLM.

3.3.2. Cơ chế Tìm kiếm lai (Hybrid Search Mechanism)

Hệ thống triển khai cơ chế tìm kiếm lai (Hybrid Search) nhằm tận dụng ưu điểm bổ trợ giữa hai phương pháp truy hồi thông tin: Dense Retrieval (Tìm kiếm ngữ nghĩa) và Sparse Retrieval (Tìm kiếm từ khóa). Mục tiêu là tối ưu hóa độ bao phủ (Recall) mà không làm giảm độ chính xác (Precision) khi xử lý các truy vấn chứa thuật ngữ chuyên ngành.

a. Kiến trúc truy hồi song song (Parallel Retrieval Architecture)

Quy trình tìm kiếm được thực hiện qua 3 giai đoạn chính:

1. Giai đoạn 1: Truy xuất ứng viên mở rộng (Candidate Expansion)

- Hệ thống thực hiện song song hai luồng tìm kiếm trên cơ sở dữ liệu Qdrant.
- Để đảm bảo không bỏ sót các kết quả tiềm năng cho bước hợp nhất, số lượng ứng viên truy xuất ở mỗi luồng được thiết lập là $2 \times Top_K$ (ví dụ: cần lấy 10 kết quả cuối cùng thì mỗi luồng sẽ lấy 20 kết quả sơ cấp).
- Luồng Dense: Sử dụng vector "dense" để tìm kiếm các đoạn văn bản có sự tương đồng về ý nghĩa.
- Luồng Sparse: Sử dụng vector "sparse" (biểu diễn trọng số BM25) để bắt chính xác các từ khóa hiếm gặp.

2. Giai đoạn 2: Hợp nhất kết quả (Score Fusion)

- Đây là thành phần cốt lõi của module. Hệ thống hỗ trợ nhiều chiến lược hợp nhất thông qua FusionStrategy Enum, trong đó Reciprocal Rank Fusion (RRF) là chiến lược mặc định và được ưu tiên sử dụng.
- Chiến lược Reciprocal Rank Fusion (RRF): RRF xếp hạng tài liệu dựa trên vị trí thứ hạng (rank) thay vì điểm số thô (score), giúp giải quyết vấn đề chênh lệch thang đo giữa Cosine Similarity (0.0 - 1.0) và BM25 Score

(không giới hạn). Công thức được cài đặt như sau:

$$Score_{RRF}(d) = \frac{1}{k + rank_{dense}(d)} + \frac{1}{k + rank_{sparse}(d)}$$

– Trong đó:

- * k : Hằng số làm mượt (Smoothing constant), được cấu hình mặc định là 60 (RRF_K=60). Giá trị này giúp giảm thiểu tác động của các tài liệu có thứ hạng quá cao ở một luồng nhưng lại quá thấp ở luồng kia.
- * $rank(d)$: Thứ hạng của tài liệu trong danh sách kết quả riêng lẻ (bắt đầu từ 1).
- Chiến lược Weighted Sum (Tổng trọng số - Tùy chọn): Trong trường hợp cần tinh chỉnh thủ công mức độ ưu tiên, hệ thống cho phép sử dụng công thức tuyến tính:

$$Score_{WS} = \alpha \cdot Score_{dense} + (1 - \alpha) \cdot Score_{sparse}$$

- Với cấu hình mặc định $\alpha = 0.7$ (HYBRID_DENSE_WEIGHT), hệ thống ưu tiên ngữ nghĩa hơn từ khóa.

3. Giai đoạn 3: Lọc và Xếp hạng lại (Filtering & Ranking)

- Danh sách kết quả sau khi hợp nhất sẽ được sắp xếp giảm dần theo điểm fused_score.
- Một bộ lọc ngưỡng (SCORE_THRESHOLD = 0.3) được áp dụng để loại bỏ các kết quả có độ tin cậy thấp trước khi trả về danh sách Top_K cuối cùng.

b. Cấu hình tham số hệ thống (System Configuration)

3.4 Chiến lược Fine-tuning mô hình Reranking

Mặc dù các mô hình ngôn ngữ tiếng Việt tiền huấn luyện (Pre-trained Models) hiện nay đã đạt hiệu suất tốt, chúng thường thiếu kiến thức đặc thù của miền dữ liệu chuyên biệt (domain-specific knowledge). Để tối ưu hóa độ chính xác ở bước xếp hạng cuối cùng, khóa luận đề xuất quy trình tinh chỉnh (fine-tuning) mô hình Reranking dựa trên kiến trúc Cross-Encoder với chiến lược khai thác mẫu khó (Hard Negatives Mining).

Bảng 3.1: Bảng cấu hình tham số kỹ thuật cho cơ chế Hybrid Search

Tham số (Parameter)	Giá trị	Giải thích kỹ thuật
ENABLE_HYBRID_SEARCH	True	Kích hoạt chế độ tìm kiếm lai. Nếu False, hệ thống sẽ chuyển về Dense Search thuần túy.
HYBRID_FUSION_METHOD	RRF	Chiến lược hợp nhất Reciprocal Rank Fusion, được chọn vì tính ổn định cao.
RRF_K	60	Hằng số làm mượt chuẩn, giúp cân bằng ảnh hưởng giữa thứ hạng cao và thấp.
SCORE_THRESHOLD	0.30	Ngưỡng điểm tối thiểu để kết quả được coi là phù hợp.
Retrieval Limit	$Top_K \times 2$	Số lượng ứng viên lấy ở mỗi luồng để đảm bảo tính toán RRF chính xác.

3.4.1. Lựa chọn Base Model (itdainb/PhoRanker)

Hệ thống lựa chọn itdainb/PhoRanker làm mô hình cơ sở. Đây là mô hình Cross-Encoder được tối ưu hóa đặc biệt cho tác vụ xếp hạng văn bản tiếng Việt.

a. Kiến trúc Cross-Encoder

Khác với Bi-Encoder (dùng để tìm kiếm nhanh ở bước 3.3), PhoRanker hoạt động theo cơ chế Cross-Encoder.

- Cơ chế: Đầu vào của mô hình là cặp chuỗi văn bản được nối với nhau: [CLS] Query [SEP] Document [SEP].
- Cơ chế Attention: Nhờ kiến trúc Transformer, cơ chế Self-Attention có thể "nhìn" thấy toàn bộ mối quan hệ giữa các từ trong câu hỏi và các từ trong tài liệu cùng một lúc (full self-attention over the pair).
- Đầu ra: Vector tại vị trí [CLS] được đưa qua một lớp tuyến tính (Linear Layer) để xuất ra một giá trị thực (logits) biểu thị độ phù hợp (relevance score).

b. Lý do lựa chọn

- Hiệu năng: Cross-Encoder cho độ chính xác cao hơn đáng kể so với Bi-Encoder vì nó nắm bắt được các sắc thái ngữ nghĩa tinh tế (như phủ định, so sánh) mà phép tính Cosine Similarity đơn giản thường bỏ qua.
- Tối ưu tài nguyên: Vì bước Reranking chỉ thực hiện trên tập nhỏ ($Top_K \approx 5 - 10$ tài liệu), chi phí tính toán cao của Cross-Encoder là chấp nhận được trong bài toán thời gian thực.

3.4.2. Phương pháp xây dựng dữ liệu huấn luyện

Chất lượng của mô hình Reranking phụ thuộc hoàn toàn vào khả năng phân biệt giữa "đúng" và "gần đúng". Hệ thống áp dụng phương pháp Sinh dữ liệu giả lập (Synthetic Data Generation) sử dụng LLM để tự động tạo ra các cặp huấn luyện chất lượng cao. Quy trình này bao gồm hai công đoạn chính: Sinh mẫu dương và Khai thác mẫu âm khó.

a. Sinh mẫu dương (Positive Samples Generation)

Hệ thống coi các đoạn văn bản (chunks) trong cơ sở dữ liệu là "chân lý" (ground truth) và sử dụng mô hình ngôn ngữ lớn để sinh ngược lại các câu hỏi tương ứng. Quy trình được thực hiện như sau:

1. Nguồn dữ liệu: Duyệt qua toàn bộ tập dữ liệu các đoạn văn bản đã được phân đoạn (chunks).
2. Đa dạng hóa truy vấn: Với mỗi đoạn văn bản d , hệ thống yêu cầu LLM sinh ra 3 biến thể câu hỏi (q_1, q_2, q_3) mà đoạn văn bản đó có thể trả lời. Để tăng tính bao quát, các chiến lược sinh câu hỏi khác nhau được áp dụng:
 - Chiến lược 1 (Direct/HyDE): Sinh câu hỏi trực tiếp dựa trên nội dung chính.
 - Chiến lược 2 (Paraphrasing): Sinh câu hỏi với văn phong khác (ví dụ: từ ngữ thông tục hoặc học thuật).
 - Chiến lược 3 (Decomposition/Complex): Sinh câu hỏi phức hợp đòi hỏi suy luận từ đoạn văn.
3. Gán nhãn: Các cặp (q_i, d) được gán nhãn là Positive (Label = 1.0).

b. Khai thác mẫu âm khó (Hard Negatives Mining)

Đây là đóng góp kỹ thuật quan trọng của hệ thống. Thay vì chọn mẫu âm ngẫu nhiên (Random Negatives - ghép câu hỏi với văn bản bất kỳ), hệ thống áp dụng chiến lược khai thác dựa trên độ tương đồng từ vựng (Lexical Similarity) sử dụng thuật toán BM25.

- Định nghĩa "Hard Negative": Là các đoạn văn bản có chứa nhiều từ khóa trùng khớp với câu hỏi (High BM25 Score) nhưng thực chất lại sai về ngữ nghĩa hoặc không chứa câu trả lời đúng.
- Mục tiêu huấn luyện: Việc ép mô hình phân biệt giữa d^+ (đúng ngữ nghĩa) và d^- (giống từ khóa nhưng sai nội dung) buộc mô hình phải học sâu hơn về ngữ cảnh thay vì chỉ bắt cặp từ khóa đơn thuần.

Quy trình khai thác (Mining Algorithm): Quy trình được thực hiện offline theo các bước sau:

1. Truy hồi ứng viên: Với mỗi câu hỏi huấn luyện q , hệ thống sử dụng BM25 để truy xuất danh sách Top_K (ví dụ: 50) đoạn văn bản có điểm số cao nhất từ kho dữ liệu.
2. Lọc chân lý (Ground-truth Filtering): Loại bỏ đoạn văn bản gốc (d^+) khỏi danh sách kết quả.
3. Tuyển chọn mẫu khó: Chọn ra N đoạn văn bản (ví dụ: Top-3) có thứ hạng cao nhất trong danh sách còn lại. Đây là những đoạn văn mà thuật toán từ khóa "nhầm tưởng" là đúng.
4. Gán nhãn: Các cặp (q, d^-) này được gán nhãn $Label = 0.0$.

c. Cấu trúc và Tỷ lệ dữ liệu

Dữ liệu cuối cùng được đóng gói dưới dạng đối tượng InputExample để đưa vào quá trình huấn luyện:

- Mẫu dương: InputExample(texts=[query, positive_chunk], label=1.0)
- Mẫu âm: InputExample(texts=[query, negative_chunk], label=0.0)

Hệ thống duy trì tỷ lệ Positive/Negative hợp lý (được theo dõi qua log positive_count/negative_count) để tránh hiện tượng mất cân bằng dữ liệu (class imbalance), đảm bảo mô hình không bị thiên lệch về một nhãn cụ thể.

3.4.3. Hàm mất mát (Loss Function) và chiến lược huấn luyện

Để tối ưu hóa mô hình Cross-Encoder cho bài toán phân loại nhị phân (liệu văn bản có phù hợp với truy vấn hay không), khóa luận sử dụng hàm mất mát chuyên dụng và chiến lược huấn luyện tối ưu hóa hiệu năng tính toán.

a. Hàm mất mát: Binary Cross-Entropy with Logits (BCEWithLogitsLoss)

Thay vì sử dụng hàm BCELoss thông thường (yêu cầu đầu vào là xác suất từ hàm Sigmoid), hệ thống sử dụng hàm BCEWithLogitsLoss của thư viện PyTorch.

- Định nghĩa toán học: Mô hình f_θ nhận đầu vào là cặp (q, d) và trả về một giá trị thực (logit) x . Hàm mất mát được định nghĩa như sau:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\sigma(x_i)) + (1 - y_i) \cdot \log(1 - \sigma(x_i))]$$

– Trong đó:

* N : Kích thước batch (Batch size).

- * y_i : Nhãn thực tế (1.0 cho Positive, 0.0 cho Negative).
- * x_i : Giá trị logit đầu ra của mô hình (trước khi qua hàm kích hoạt).
- * $\sigma(x)$: Hàm Sigmoid, $\sigma(x) = \frac{1}{1+e^{-x}}$.

- Biện luận kỹ thuật (Technical Justification): Việc lựa chọn BCEWithLogitsLoss thay vì tách rời Sigmoid và BCELoss mang lại các lợi điểm kỹ thuật quan trọng:

1. Tính ổn định số học (Numerical Stability): Hàm này thực hiện kết hợp lớp Sigmoid và hàm log loss vào một lớp duy nhất. Nhờ sử dụng thuật toán LogSumExp, nó tránh được các lỗi tràn số (overflow) hoặc dưới mức (underflow) khi giá trị logit x quá lớn hoặc quá nhỏ (extreme values).
2. Xử lý Logits thô: Mô hình không cần tốn tài nguyên tính toán để chuẩn hóa đầu ra thành xác suất trong quá trình huấn luyện, mà trực tiếp sử dụng giá trị thô (logits) từ lớp Linear cuối cùng.

b. Chiến lược huấn luyện (Training Strategy)

Quá trình huấn luyện mô hình Reranking được thực hiện theo phương pháp Tinh chỉnh toàn phần (Full Fine-tuning), cập nhật trọng số của toàn bộ 135 triệu tham số thay vì sử dụng các phương pháp PEFT (như LoRA). Để thực hiện điều này trên tài nguyên phần cứng giới hạn, hệ thống áp dụng các chiến lược tối ưu hóa bộ nhớ và lịch trình học tập thích ứng.

1. Tối ưu hóa Bộ nhớ và Tốc độ (Memory & Speed Optimization) Để khắc phục giới hạn VRAM, hệ thống kết hợp hai kỹ thuật:

- Tích lũy Gradient (Gradient Accumulation): Thay vì nạp một batch lớn vào bộ nhớ, hệ thống chia nhỏ thành các micro-batches kích thước 4. Gradient được tính toán và tích lũy qua 4 bước trước khi thực hiện một lần cập nhật trọng số (optimizer step).

- Hiệu quả: Giả lập kích thước batch hiệu dụng (Effective Batch Size) là $4 \times 4 = 16$. Điều này đảm bảo độ ổn định của Gradient (giảm nhiễu) mà không gây tràn bộ nhớ (OOM).

- Độ chính xác hỗn hợp (Mixed Precision Training - FP16): Sử dụng torch.cuda.amp.Grad để thực hiện các phép toán nhân ma trận dưới định dạng 16-bit (FP16) trong khi lưu trữ trọng số chính ở 32-bit (FP32). Kỹ thuật này giảm 50% lượng bộ nhớ tiêu thụ và tăng tốc độ huấn luyện.

2. Bộ tối ưu hóa và Lịch trình học (Optimizer & Scheduler)

- Optimizer: Sử dụng thuật toán AdamW (Adam with Weight Decay). Đây

là chuẩn mực cho việc fine-tune các mô hình ngôn ngữ, giúp mô hình hội tụ nhanh đồng thời áp dụng phân rã trọng số (weight decay) để giảm thiểu overfitting.

- Learning Rate Schedule: Áp dụng chiến lược Linear Warmup with Linear Decay:

- Giai đoạn Warmup (10% số bước đầu): Tăng tuyến tính tốc độ học từ 0 lên $2e - 5$. Giai đoạn này giúp "làm nóng" các tham số, tránh gây sốc (shock) cho mô hình pre-trained bởi các gradient lớn ban đầu.
- Giai đoạn Decay (90% số bước sau): Giảm tuyến tính tốc độ học từ $2e - 5$ về 0. Điều này giúp mô hình tinh chỉnh các chi tiết nhỏ (fine-grained patterns) khi tiến dần đến điểm cực trị địa phương.

3. Cơ chế kiểm soát và Dừng sớm (Regularization & Early Stopping) Để đảm bảo mô hình có khả năng tổng quát hóa tốt (generalization) và tránh overfitting trên tập train:

- Đánh giá liên tục: Thực hiện kiểm thử trên tập Validation (10% dữ liệu) sau mỗi 100 bước huấn luyện.
- Lưu trữ Checkpoint: Chỉ lưu giữ 2 phiên bản mô hình có kết quả đánh giá tốt nhất
- Early Stopping: Cơ chế dừng sớm được kích hoạt nếu chỉ số đánh giá (Evaluation Loss) không cải thiện sau 2 lần kiểm thử liên tiếp

Bảng 3.2: Bảng tham số cấu hình chi tiết cho quá trình huấn luyện

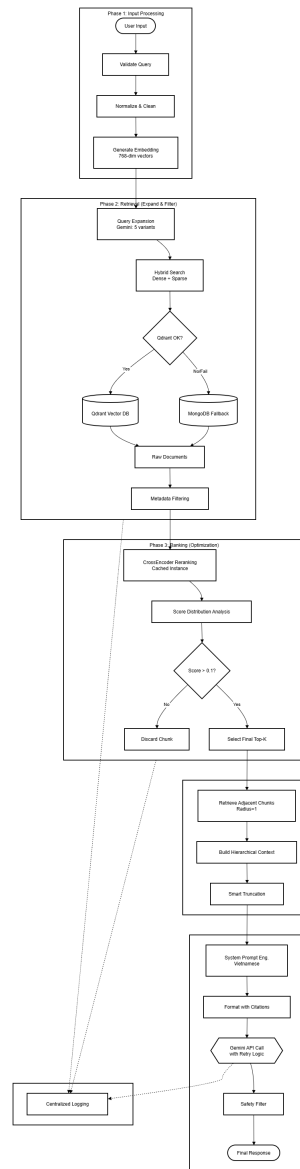
Tham số (Parameter)	Giá trị	Giải thích kỹ thuật
learning_rate	2×10^{-5}	Tốc độ học tiêu chuẩn cho việc tinh chỉnh (fine-tuning) các mô hình Transformer.
num_epochs	4	Số vòng lặp huấn luyện tối đa, được giới hạn để tránh hiện tượng quá khớp (overfitting).
train_batch_size	4	Kích thước batch vật lý trên mỗi thiết bị (GPU) để phù hợp với bộ nhớ VRAM giới hạn.
gradient_accumulation	4	Số bước tích lũy gradient. Tạo ra kích thước batch hiệu dụng (Effective Batch Size) là $4 \times 4 = 16$.
warmup_ratio	0.1	Dành 10% số bước đầu tiên để tăng dần tốc độ học (Linear Warmup), giúp ổn định mô hình.
fp16 (Mixed Precision)	True	Sử dụng độ chính xác hỗn hợp (16-bit) để giảm tiêu thụ bộ nhớ và tăng tốc độ tính toán.
evaluation_strategy	Steps (100)	Thực hiện đánh giá trên tập validation và lưu checkpoint sau mỗi 100 bước.
early_stopping_patience	2	Dừng huấn luyện sớm nếu kết quả đánh giá không cải thiện sau 2 lần kiểm tra liên tiếp.

3.5 Module sinh câu trả lời

Module Sinh câu trả lời là thành phần cuối cùng trong kiến trúc RAG, chịu trách nhiệm tổng hợp thông tin từ các đoạn văn bản đã được tái xếp hạng (Reranked Context) để tạo ra phản hồi cho người dùng. Mục tiêu của module này không chỉ là trả lời đúng câu hỏi mà còn phải đảm bảo tính trung thực (faithfulness), khả năng truy xuất nguồn gốc (traceability) và giảm thiểu ảo giác (hallucination).

3.5.1. Kiến trúc luồng xử lý dữ liệu (Generation Pipeline)

Luồng xử lý được tổ chức theo mô hình đường ống tuần tự (Sequential Pipeline), trong đó dữ liệu đi qua các cổng kiểm soát (gates) nghiêm ngặt trước khi đến được mô hình sinh.



Hình 3.1: Kiến trúc tổng quan luồng xử lý dữ liệu

a. Xử lý đầu vào (Input Processing)

Trước khi đi vào các luồng tìm kiếm phức tạp, câu hỏi thô của người dùng (Raw Query) phải trải qua một lớp chuẩn hóa nghiêm ngặt. Mục tiêu là loại bỏ nhiễu, đảm bảo tính nhất quán về định dạng và tối ưu hóa khả năng so khớp với dữ liệu trong Vector DB. Quy trình bao gồm 5 bước tuần tự:

1. Chuẩn hóa khoảng trắng (Whitespace Normalization)

- Thao tác: Loại bỏ khoảng trắng thừa ở đầu/cuối câu (Trim/Strip) và hợp nhất các khoảng trắng lặp lại ở giữa câu (Collapse Spaces).

- Mục đích: Người dùng thường vô tình nhập thừa dấu cách hoặc xuống dòng. Việc làm sạch này giúp giảm chiều dài chuỗi ký tự không cần thiết và tránh làm nhiễu mô hình Tokenizer.

2. Chuẩn hóa Unicode (Unicode Normalization)

- Thao tác: Chuyển đổi toàn bộ chuỗi ký tự về dạng chuẩn Unicode.
- Mục đích: Đây là bước tối quan trọng đối với tiếng Việt để xử lý sự không nhất quán giữa kiểu gõ "Dừng sẵn" và "Tổ hợp". Nếu không chuẩn hóa, từ "Mô" gõ theo hai kiểu khác nhau sẽ có mã nhị phân khác nhau, dẫn đến việc tìm kiếm thất bại.

3. Loại bỏ ký tự đặc biệt (Character Sanitization)

- Thao tác: Lọc bỏ các dấu câu không mang nghĩa ngữ pháp quan trọng hoặc các ký tự nhiễu (như @, #, * nếu không cần thiết).
- Mục đích: Giúp mô hình tập trung vào nội dung ngữ nghĩa chính (keywords) thay vì bị phân tâm bởi các ký tự cú pháp nhiễu.

b. Mã hóa Vector truy vấn

Sau bước làm sạch sơ bộ, câu hỏi dạng văn bản tự nhiên được chuyển đổi thành biểu diễn vector (dense vector representation) để thực hiện các phép toán so khớp trong không gian chiều cao.

- Thay vì sử dụng các mô hình đa ngôn ngữ phổ thông (như paraphrase-multilingual-MiniLM-L12-v2), hệ thống tích hợp mô hình `dangvantuan/vietnamese-document-embedding`.
- Kiến trúc: Dựa trên nền tảng RoBERTa (Robustly optimized BERT approach).
- Đặc điểm: Đây là mô hình đơn ngữ (monolingual) được huấn luyện chuyên sâu trên ngữ liệu tiếng Việt lớn, cho khả năng nắm bắt các đặc trưng ngôn ngữ phức tạp (từ ghép, thành ngữ, cấu trúc đảo ngữ) tốt hơn so với các mô hình đa ngôn ngữ cùng kích thước.

3.5.2. Chiến lược Truy hồi nâng cao (Advanced Retrieval Strategy)

Để tối đa hóa độ bao phủ (Recall), hệ thống không chỉ tìm kiếm dựa trên câu hỏi gốc mà triển khai quy trình Truy hồi Đa biến thể (Multi-variant Retrieval). Quy trình này được thiết kế đặc biệt để vượt qua các rào cản về bộ lọc nội dung của LLM và sự đa dạng ngôn ngữ tự nhiên.

a. Mở rộng Truy vấn An toàn (Safety-Aware Query Expansion)

Do đặc thù của miền dữ liệu (có thể chứa các thuật ngữ quân sự hoặc an ninh mạng như "tấn công", "tiêu diệt", "vũ khí"), việc gửi trực tiếp câu hỏi gốc lên các LLM thương mại (như Gemini) thường kích hoạt bộ lọc an toàn (Safety Filters), dẫn đến việc từ chối phản hồi. Hệ thống giải quyết vấn đề này bằng cơ chế Khử nhạy cảm Tiền xử lý (Pre-expansion Sanitization)

- Từ điển thay thế (Sanitization Dictionary): Hệ thống duy trì một bảng ánh xạ các từ khóa nhạy cảm sang các từ trung tính trước khi gửi prompt mở rộng.
- Prompting: Câu hỏi sau khi làm sạch (safe_question) được đưa vào Gemini-2.5-flash với yêu cầu sinh ra 5 biến thể (num_variants=5) giữ nguyên ý nghĩa nhưng thay đổi cấu trúc câu.
- Phân tích (Parsing): Kết quả trả về được phân tích bằng biểu thức chính quy (Regex) để trích xuất sạch các dòng bắt đầu bằng số thứ tự, loại bỏ các lời dẫn dắt thừa của AI.

b. Chiến lược Truy hồi Lặp (Iterative Retrieval Strategy)

Thay vì nối tất cả các biến thể thành một câu truy vấn dài (gây loãng vector), hệ thống thực hiện tìm kiếm độc lập cho từng biến thể.

- Quy trình: Với tập truy vấn $Q = \{q_{original}, q_{var1}, \dots, q_{var4}\}$:
 1. Mỗi $q_i \in Q$ được mã hóa thành vector riêng biệt.
 2. Thực hiện Hybrid Search (kết hợp Dense + Sparse) cho từng q_i để thu về tập kết quả cục bộ R_i .
 3. Cơ chế Fallback được áp dụng độc lập: nếu truy vấn vector cho q_i thất bại, hệ thống tự động chuyển sang tìm kiếm từ khóa cho q_i đó.

c. Hợp nhất và Khử trùng lặp (Aggregation & Deduplication)

Kết quả cuối cùng là hợp (Union) của tất cả các kết quả cục bộ: $R_{final} = \bigcup R_i$.

- Deduplication: Sử dụng tập hợp HashSet để đảm bảo mỗi tài liệu chỉ xuất hiện một lần duy nhất trong danh sách.
- Sắp xếp toàn cục: Toàn bộ danh sách kết quả duy nhất được sắp xếp lại dựa trên điểm số giảm dần.
- Cắt ngưỡng (Slicing): Chỉ giữ lại Top_K kết quả xuất sắc nhất để chuyển sang giai đoạn Tái xếp hạng (Reranking).

3.5.3. Tối ưu hóa Reranking và Mở rộng ngữ cảnh (Reranking & Context Optimization)

Để đảm bảo chất lượng đầu vào tốt nhất cho mô hình sinh (LLM), hệ thống thực hiện hai bước hậu xử lý (post-processing) quan trọng: tái xếp hạng với mô hình Cross-Encoder và khôi phục ngữ cảnh bị mất do quá trình phân đoạn.

a. Tối ưu hóa Reranking

Sau khi nhận được danh sách ứng viên từ bước tìm kiếm sơ cấp, hệ thống kích hoạt quy trình tái xếp hạng sử dụng mô hình Cross-Encoder. Khác với Bi-Encoder (tìm kiếm nhanh), Cross-Encoder so sánh trực tiếp cặp câu (Query, Document) để tính điểm phù hợp chính xác hơn. Các kỹ thuật tối ưu hiệu năng:

1. Cơ chế Caching Instance:

- Việc khởi tạo Cross-Encoder tốn tài nguyên (1-3 giây). Hệ thống sử dụng mẫu thiết kế Singleton để lưu trữ instance trong bộ nhớ cache, loại bỏ độ trễ khởi động cho các request tiếp theo.

2. Xử lý Batch và Cắt gọt (Batching & Truncation):

- Để tránh lỗi tràn bộ nhớ GPU (OOM) và cảnh báo tokenizer, văn bản đầu vào được cắt ngắn ở mức 1500 ký tự (400 tokens).
- Dữ liệu được xử lý theo lô (batch_size=32), tận dụng khả năng tính toán song song của GPU.

3. Lọc ngưỡng mềm (Soft Thresholding):

- Hệ thống áp dụng ngưỡng sàn `min_rerank_score = 0.1`.
- Logic: Các đoạn văn bản có điểm số thấp hơn 0.1 được coi là nhiễu và bị loại bỏ ngay lập tức, giúp giảm tải lượng token không cần thiết gửi đến LLM.

4. Giám sát thống kê (Statistical Monitoring):

- Hệ thống tính toán các chỉ số phân phối điểm số (min, max, mean, std) ngay trong luồng xử lý. Điều này giúp phát hiện sớm các bất thường (ví dụ: mô hình bị trôi/drift hoặc chất lượng truy vấn kém).

b. Chiến lược Mở rộng Ngữ cảnh

Một hạn chế cố hữu của RAG là việc phân đoạn (chunking) thường cắt đứt mạch văn, khiến các đại từ nhân xưng hoặc mối quan hệ nguyên nhân-kết quả bị tách rời. Hệ thống khắc phục bằng thuật toán Truy hồi lân cận (Adjacent Chunk Retrieval). Thuật toán mở rộng:

1. Xác định hạt nhân: Chọn N kết quả tốt nhất (Top-N) sau bước Reranking làm các "hạt nhân" ngữ cảnh.
2. Truy vấn bán kính (Radius Query):
 - Với mỗi chunk hạt nhân có chỉ số thứ tự là i , hệ thống truy vấn cơ sở dữ liệu MongoDB để lấy các chunk lân cận tại vị trí $i - 1$ (Previous) và $i + 1$ (Next).
3. Hợp nhất thông minh:
 - Kiểm tra trùng lặp để đảm bảo không lấy lại các chunk đã tồn tại trong danh sách kết quả chính.
 - Gắn nhãn metadata: Các chunk gốc được đánh dấu là main, chunk mở rộng là expanded để phục vụ việc hiển thị nguồn trích dẫn sau này.

Ý nghĩa kỹ thuật: Việc ghép nối này ($Chunk_{prev} + Chunk_{main} + Chunk_{next}$) giúp tái tạo lại một "cửa sổ ngữ cảnh" liền mạch, cung cấp cho Gemini thông tin nền đầy đủ để trả lời các câu hỏi phức tạp đòi hỏi sự liên kết logic.

3.5.4. Kỹ thuật Prompt Engineering và Cơ chế An toàn (Prompting & Safety)

Để đảm bảo chất lượng câu trả lời từ mô hình ngôn ngữ (LLM) đồng thời tuân thủ các nguyên tắc an toàn nội dung, hệ thống áp dụng một chiến lược Prompt Engineering hai lớp: Lớp Chỉ thị Tiêu chuẩn (Standard Instruction) và Lớp Khử nhạy cảm (Sanitization Layer).

a. Thiết kế System Prompt Tiêu chuẩn (Standard Instruction Design)

Đối với các truy vấn thông thường, hệ thống sử dụng một bộ chỉ thị (System Message) nghiêm ngặt gồm 7 quy tắc cốt lõi, nhằm định hình hành vi của trợ lý AI:

1. Định danh (Persona): "Trợ lý AI thông minh chuyên phân tích tài liệu kỹ thuật".
2. Tính trung thực (Grounding): Yêu cầu CHỈ sử dụng thông tin có trong ngữ cảnh. Nếu không tìm thấy, bắt buộc phải trả lời "Không tìm thấy thông tin" thay vì tự bịa đặt (Hallucination).
3. Định dạng trích dẫn (Citation Format): Bắt buộc trích dẫn nguồn theo định dạng chuẩn [document_id tr.X]. Điều này giúp người dùng dễ dàng truy xuất ngược lại vị trí thông tin gốc trong tài liệu PDF/Word.
4. Ngôn ngữ: Ưu tiên độ chính xác hơn độ dài, trả lời bằng tiếng Việt súc tích.

b. Cơ chế Khử nhạy cảm Động (Dynamic Sanitization Mechanism)

Khi xử lý các tài liệu thuộc miền dữ liệu đặc thù (như an ninh quốc phòng, an toàn thông tin, hoặc lịch sử xung đột), các từ khóa chuyên ngành thường kích hoạt nhầm bộ lọc an toàn (Safety Filters) của các LLM thương mại, dẫn đến việc từ chối trả lời. Hệ thống giải quyết vấn đề bằng thực hiện các bước sau trước khi gửi đến API:

- **Từ điển Thay thế (Replacement Dictionary):** Hệ thống duy trì một bảng ánh xạ các từ khóa "nhạy cảm" sang các từ khóa "trung tính" (Neutral Synonyms) bằng kỹ thuật Regex (Case-insensitive).
 - Mục đích: Giữ nguyên cấu trúc ngữ nghĩa của câu chuyện nhưng giảm thiểu "độ nóng" (toxicity score) của văn bản đầu vào.
- **Chuyển đổi System Prompt (Prompt Switching):** Khi phát hiện ngữ cảnh nhạy cảm, hệ thống tự động chuyển từ prompt "Chuyên gia kỹ thuật" sang prompt "Trợ lý hỗ trợ thông tin" với giọng văn khách quan hơn, giúp tăng tỷ lệ được chấp nhận (acceptance rate) từ phía LLM.
- **Cắt gọt Ngữ cảnh An toàn (Context Truncation):** Ngữ cảnh đầu vào được giới hạn cứng ở mức SAFE_CONTEXT_CHARS (khoảng 2500 ký tự). Việc này không chỉ giúp tiết kiệm chi phí token mà còn giảm xác suất xuất hiện các cụm từ vi phạm chính sách trong một cửa sổ ngữ cảnh quá dài.

Chương 4: Thực nghiệm và cài đặt

4.1 Môi trường thực nghiệm và công nghệ sử dụng

- Quá trình xây dựng và đánh giá hệ thống được thực hiện trên máy laptop sử dụng hệ điều hành Windows 11. Cấu hình phần cứng bao gồm bộ vi xử lý Intel Core i5-12500H, bộ nhớ trong 16GB RAM và card đồ họa NVIDIA GeForce RTX 3050 (4GB VRAM) nhằm đáp ứng yêu cầu tính toán song song cho quá trình huấn luyện và suy luận mô hình Deep Learning.
- Về nền tảng công nghệ, hệ thống được phát triển bằng ngôn ngữ Python 3.12.7. Các thư viện nòng cốt bao gồm PyTorch và Hugging Face Transformers để quản lý mô hình ngôn ngữ và xây dựng luồng xử lý RAG (Retrieval-Augmented Generation). Cơ sở dữ liệu vector Qdrant được sử dụng để lưu trữ và truy vấn ngữ nghĩa, trong khi MongoDB đóng vai trò lưu trữ đối tượng (Object Storage) cho các tài liệu gốc. Toàn bộ môi trường ứng dụng được quản lý thông qua Docker để đảm bảo tính nhất quán khi triển khai.

4.2 Quy trình xử lý dữ liệu

Dữ liệu đóng vai trò cốt lõi trong việc xây dựng cơ sở tri thức (Knowledge Base) và đánh giá hiệu năng của hệ thống. Phần này trình bày chi tiết về nguồn dữ liệu gốc được sử dụng để huấn luyện/nhúng (embedding) và quy trình xây dựng bộ dữ liệu chuẩn (Golden Dataset) phục vụ cho quá trình kiểm thử.

4.2.1. Dữ liệu gốc (Tài liệu môn học)

- Dữ liệu đầu vào cho hệ thống là tập hợp các tài liệu chính thống của môn: Công nghệ phần mềm, mạng máy tính, cơ sở dữ liệu. Các tài liệu này được thu thập dưới dạng văn bản phi cấu trúc (unstructured text) từ các nguồn bài giảng và giáo trình. Dữ liệu gốc ở định dạng pdf.
- Quy trình tiền xử lý (Preprocessing): Do đặc thù của dữ liệu thô chứa nhiều nhiễu (header, footer, số trang, hình ảnh không có chú thích...), quy trình tiền xử lý được thực hiện qua các bước sau:
 1. Trích xuất văn bản (Text Extraction): Sử dụng thư viện để chuyển đổi nội dung từ PDF/Slide sang dạng văn bản thuần (plain text).
 2. Làm sạch (Cleaning): Loại bỏ các ký tự đặc biệt, khoảng trắng thừa, và các thông tin không mang ngữ nghĩa (như tiêu đề lặp lại ở chân trang).
 3. Phân đoạn (Chunking): Đây là bước quan trọng nhất đối với mô hình RAG. Văn bản dài được chia thành các đoạn nhỏ (chunks) để tối ưu hóa

khả năng tìm kiếm vector.

4.2.2. Xây dựng bộ dữ liệu đánh giá chuẩn (Golden Dataset/QA_Gold)

Để đánh giá khách quan độ chính xác của hệ thống trong cả hai tác vụ Truy xuất (Retrieval) và Sinh câu trả lời (Generation), một bộ dữ liệu chuẩn (Golden Dataset) đã được xây dựng bán tự động. Bộ dữ liệu này đóng vai trò là "chân lý" (Ground Truth) để so sánh kết quả đầu ra của mô hình.

a. Quy trình xây dựng dữ liệu (Data Construction Pipeline)

Quy trình xây dựng bộ dữ liệu được thực hiện theo phương pháp "Bán tự động có kiểm soát" (Human-in-the-loop), bao gồm 4 bước chính:

1. Trích chọn ngữ cảnh (Context Sampling):

- Từ tập dữ liệu gốc đã được phân đoạn (chunking), hệ thống lựa chọn ngẫu nhiên các đoạn văn bản (chunks) nhưng đảm bảo độ bao phủ trên toàn bộ các chương mục của môn học.
- Loại bỏ các đoạn văn bản quá ngắn (dưới 50 từ) hoặc thiếu ngữ nghĩa (ví dụ: mục lục, danh sách tài liệu tham khảo).

2. Sinh câu hỏi giả định (Question Generation):

- Với mỗi đoạn văn bản được chọn, ta thực hiện đặt câu hỏi cho LLM dựa trên nội dung cốt lõi của đoạn đó.
- Nguyên tắc: Câu hỏi phải trả lời được chỉ bằng thông tin có trong đoạn văn bản đó (Closed-domain QA), tránh việc mô hình phải sử dụng kiến thức bên ngoài (hallucination).

3. Soạn thảo câu trả lời chuẩn (Answer Annotation):

- Câu trả lời (Ground Truth) được soạn thảo thủ công để đảm bảo súc tích, đi thẳng vào vấn đề.
- Đáp án không được sao chép nguyên văn (copy-paste) 100% từ tài liệu mà cần được viết lại (paraphrase) để kiểm tra khả năng hiểu ngôn ngữ của mô hình.

4. Kiểm định chéo (Cross-Validation):

- Bộ dữ liệu sau khi soạn thảo được rà soát lại lần cuối để loại bỏ các câu hỏi mơ hồ hoặc đáp án sai lệch kiến thức chuyên ngành.

b. Cấu trúc mẫu dữ liệu (Data Schema)

- Mỗi mẫu dữ liệu trong tập đánh giá chuẩn được lưu trữ dưới dạng đối tượng JSON (JavaScript Object Notation), bao gồm đầy đủ thông tin về câu hỏi, đáp

án chuẩn, và tham chiếu đến tài liệu gốc. Cấu trúc này giúp thuận tiện cho việc truy xuất và đánh giá tự động.

- `_id` và `question_id`: Mã định danh duy nhất (Unique Identifier) của cặp câu hỏi - câu trả lời, dùng để quản lý, truy vết và tham chiếu trong quá trình đánh giá.
- `document_id`: Mã định danh của tài liệu gốc (Source Document). Trường này giúp liên kết câu hỏi với tài liệu nguồn cụ thể (ví dụ: một file PDF giáo trình hoặc một slide bài giảng).
- `question_text`: Nội dung văn bản của câu hỏi (User Query). Đây là đầu vào giả lập cho hệ thống, mô phỏng các thắc mắc thực tế của người học.
- `gold_answer`: Câu trả lời chuẩn (Ground Truth Answer). Đây là đáp án chính xác được biên soạn và kiểm duyệt bởi con người, đóng vai trò là tiêu chuẩn để chấm điểm khả năng sinh văn bản (Generation) của mô hình.
- `gold_support_chunk_ids`: Mã định danh của đoạn văn bản chứa thông tin trả lời (Supporting Chunk/Context). Trường này được sử dụng làm "chân lý" để đánh giá độ chính xác của module truy xuất (Retriever) – xác định xem hệ thống có tìm đúng vị trí kiến thức hay không.
- `type`: Nhãn phân loại câu hỏi (ví dụ: Fact - Sự kiện, Reasoning - Suy luận). Trường này hỗ trợ việc phân tích chi tiết hiệu năng của mô hình trên từng nhóm câu hỏi khác nhau.

4.3 Quá trình Fine-tuning PhoRanker

Để nâng cao khả năng xếp hạng văn bản tiếng Việt, mô hình PhoRanker (được phát triển dựa trên kiến trúc [ví dụ: PhoBERT-base/RoBERTa]) đã được tinh chỉnh (fine-tune) trên tập dữ liệu chuyên ngành. Quá trình này giúp mô hình học được các đặc trưng ngữ nghĩa cụ thể của miền dữ liệu môn học thay vì chỉ sử dụng tri thức phổ quát ban đầu.

4.3.1. Thiết lập tham số huấn luyện (Hyperparameters)

- Mô hình nền tảng được lựa chọn để tinh chỉnh là `itdainb/PhoRanker`, một mô hình Cross-Encoder chuyên dụng cho tiếng Việt. Quá trình huấn luyện được thực hiện theo phương pháp Full Fine-tuning (tinh chỉnh toàn bộ trọng số) thay vì sử dụng LoRA, nhằm đạt được hiệu năng tối đa cho tác vụ xếp hạng.
- Để giải quyết bài toán tài nguyên phần cứng giới hạn (GPU VRAM thấp) nhưng vẫn đảm bảo sự ổn định của quá trình hội tụ, kỹ thuật Gradient Accumulation

và Mixed Precision (FP16) đã được áp dụng. Cụ thể, kích thước batch vật lý là 4, kết hợp với 4 bước tích lũy gradient, tạo ra kích thước batch hiệu dụng (Effective Batch Size) là 16.

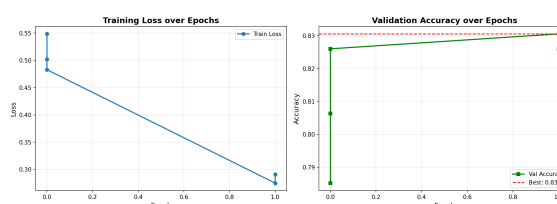
- Dữ liệu huấn luyện sử dụng chiến lược Hard Negatives (các mẫu âm tính khó phân biệt) được sinh ra từ thuật toán BM25, giúp mô hình học cách phân biệt các văn bản có từ khóa giống nhau nhưng khác ngữ nghĩa.
- Chi tiết các siêu tham số được trình bày trong bảng sau:

Bảng 4.1: Siêu tham số huấn luyện mô hình PhoRanker

Tham số	Giá trị	Giải thích kỹ thuật
Base Model	itdainb/PhoRanker	Cross-Encoder kiến trúc Roberta, tối ưu cho tiếng Việt.
Loss Function	BCEWithLogitsLoss	Hàm mất mát Binary Cross Entropy, phù hợp cho bài toán phân loại nhị phân (liên quan/không liên quan).
Batch Size	16 (Effective)	Batch size thực tế mỗi thiết bị là 4, tích lũy gradient qua 4 bước (Gradient Accumulation Steps = 4).
Epochs	4	Số vòng lặp huấn luyện, sử dụng chiến lược Early Stopping (patience=2) để chống overfitting.
Learning Rate	2e-5	Tốc độ học tiêu chuẩn cho fine-tuning, kết hợp Warmup Ratio 0.1 (10% đầu giai đoạn).
Precision	FP16 (Mixed)	Sử dụng độ chính xác hỗn hợp để giảm tiêu thụ VRAM và tăng tốc độ tính toán.
Data Split	90/10	Tỷ lệ tập Train/Validation là 9:1.

4.3.2. Biểu đồ huấn luyện và Giám sát hiệu năng

Quá trình huấn luyện được giám sát chặt chẽ thông qua việc theo dõi các chỉ số đánh giá sau mỗi 100 bước (steps). Hệ thống tự động lưu trữ (Checkpointing) trạng thái của mô hình và bộ tối ưu hóa, chỉ giữ lại 2 checkpoint tốt nhất để tiết kiệm dung lượng ổ cứng.



Hình 4.1: Biểu đồ huấn luyện

Từ biểu đồ thực nghiệm, có thể rút ra các nhận xét quan trọng về hiệu năng của mô hình PhoRanker sau quá trình tinh chỉnh:

1. Hội tụ nhanh chóng (Rapid Convergence):

- Training Loss (Biểu đồ trái): Giá trị hàm mất mát giảm mạnh và nhất quán từ mức 0.55 xuống còn 0.27 chỉ trong vòng 1 epoch. Độ dốc lớn của đường biểu diễn cho thấy mô hình thích nghi rất nhanh với dữ liệu miền chuyên ngành và học được các đặc trưng quan trọng từ các mẫu Hard Negatives.
- Điều này chứng tỏ tốc độ học (Learning Rate $2e-5$) và kích thước batch được thiết lập hợp lý, giúp mô hình không bị dao động (oscillation) quá mức.

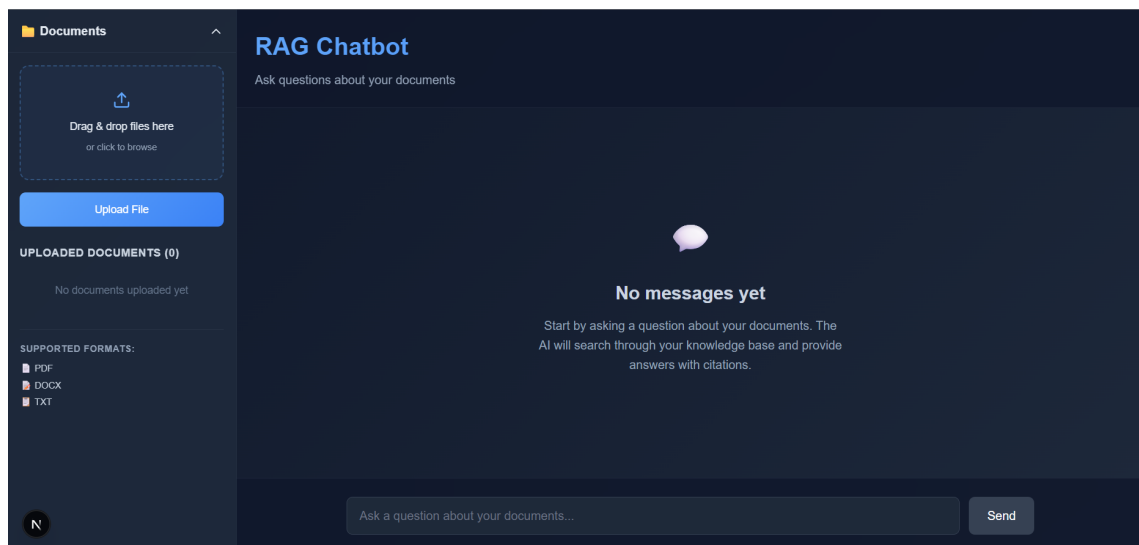
2. Hiệu suất trên tập kiểm thử (Validation Performance):

- Validation Accuracy (Biểu đồ phải): Độ chính xác tăng vọt ngay từ những bước đầu tiên, từ mức khởi điểm 78.5
- Kết quả tốt nhất (Best Metric): Mô hình đạt độ chính xác cao nhất (Best Accuracy) là 83.06% (được đánh dấu bởi đường kẻ đỏ đứt nét). Đây là một kết quả rất tích cực đối với bài toán xếp hạng văn bản tiếng Việt chuyên ngành.

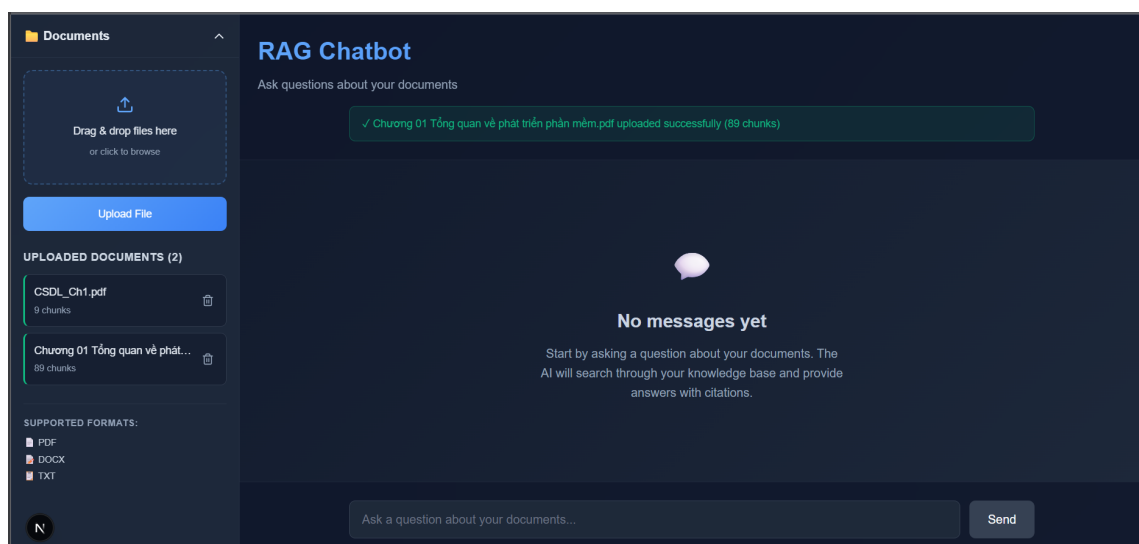
3. Dấu hiệu ổn định:

- Mặc dù có một sự sụt giảm nhẹ về độ chính xác ở cuối chu kỳ (từ 0.83 xuống khoảng 0.826), nhưng nhìn chung đường biểu diễn vẫn duy trì ở mức cao ($>82\%$). Việc lưu checkpoint dựa trên Best Validation Loss sẽ đảm bảo hệ thống luôn chọn được phiên bản mô hình tối ưu nhất (tại đỉnh 83.06%) để đưa vào sử dụng thực tế.

4.4 Demo minh họa

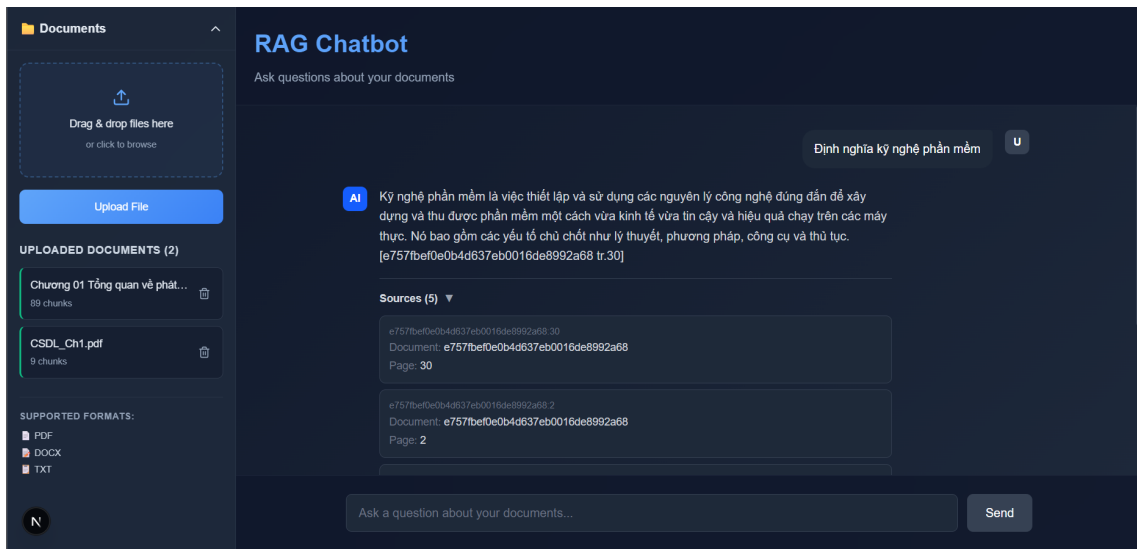


Hình 4.2: Giao diện trang chủ của hệ thống RAG Chatbot.



Hình 4.3: Người dùng có thể upload tài liệu của mình lên hệ thống

Để kiểm chứng khả năng hoạt động của hệ thống, ta thực hiện đưa ra câu hỏi liên quan về tài liệu môn học, yêu cầu khả năng tổng hợp thông tin.



Hình 4.4: Giao diện đưa ra câu trả lời

Kết quả trả về bao gồm:

- Câu trả lời trực tiếp: Ngắn gọn, súc tích, giải thích đúng trọng tâm
- Trích dẫn nguồn (Citations): Hệ thống hiển thị rõ thông tin này được lấy từ Tài liệu nào, Trang số mấy. Đây là tính năng quan trọng giúp người dùng kiểm chứng độ tin cậy

4.5 Kết quả thực nghiệm

Kết quả thực nghiệm được tổng hợp từ 48 phiên kiểm thử (test runs) trên bộ dữ liệu Golden Dataset. Hệ thống được đánh giá trên 3 khía cạnh: Khả năng truy xuất (Retrieval), Chất lượng văn bản sinh ra (Generation), và Đánh giá định tính bởi LLM (LLM-as-a-Judge).

4.4.1. Đánh giá khả năng Truy xuất (Retrieval Performance)

Khả năng tìm kiếm tài liệu của hệ thống (sử dụng PhoRanker) được đo lường thông qua các chỉ số Recall, MRR (Mean Reciprocal Rank) và NDCG tại top 5 và top 10 kết quả. Nhận xét:

Bảng 4.2: Hiệu năng truy xuất thông tin (Retrieval Metrics)

Chỉ số (Metric)	@Top 5	@Top 10
Recall (Độ phủ)	0.604	0.792
MRR (Thứ hạng trung bình)	0.420	0.447
NDCG (Độ lợi thông tin)	0.466	0.528

- Recall@10 đạt 0.792: Điều này cho thấy trong gần 80% các trường hợp, đoạn văn bản chứa đáp án đúng (Gold Chunk) đã xuất hiện trong 10 kết quả đầu

tiên trả về. Đây là tiền đề quan trọng để module Generation có đủ ngữ cảnh trả lời.

- **MRR@5 đạt 0.420:** Giá trị này chỉ ra rằng, trung bình đoạn văn bản đúng thường xuất hiện ở vị trí thứ 2 hoặc 3 trong danh sách xếp hạng, chứng tỏ hiệu quả của mô hình Reranker trong việc đẩy tin quan trọng lên đầu.

4.4.2. Đánh giá khả năng Sinh văn bản (Generation Quality)

Chất lượng câu trả lời của Chatbot được so sánh với đáp án chuẩn (Ground Truth) thông qua các độ đo mức độ trùng khớp văn bản. Phân tích: Chỉ số ROUGE-1 đạt

Bảng 4.3: Chất lượng văn bản sinh ra (Generation Metrics)

Chỉ số	Giá trị trung bình	Ý nghĩa
ROUGE-1	0.605	Mức độ trùng khớp về từ đơn (unigram), phản ánh độ chính xác về nội dung.
ROUGE-L	0.408	Mức độ trùng khớp về cấu trúc câu dài nhất, phản ánh độ trôi chảy.
Token F1	0.409	Sự cân bằng giữa độ chính xác (Precision) và độ phủ (Recall) của các token.

0.605 là một kết quả cao đối với các tác vụ sinh văn bản tiếng Việt chuyên ngành, cho thấy Chatbot nắm bắt tốt các từ khóa quan trọng trong đáp án chuẩn.

4.4.3. Đánh giá định tính (LLM-as-a-Judge)

Bên cạnh các chỉ số kỹ thuật, hệ thống sử dụng mô hình ngôn ngữ lớn (Gemini) đóng vai trò giám khảo để chấm điểm chất lượng câu trả lời trên thang điểm 1-5 (dựa trên tiêu chí: tính hữu ích, độ chính xác và sự mạch lạc).

- **Kết quả tổng quan:**
 - Điểm trung bình (Mean Rating): 4.04 / 5.0
 - Phân bố điểm số:
 - * Điểm 5 (Xuất sắc): 21 mẫu (43.75%)
 - * Điểm 4 (Tốt): 14 mẫu (29.1%)
 - * Điểm 3 (Khá): 10 mẫu (20.8%)
 - * Điểm 1 (Kém): 3 mẫu (6.25%)

Kết quả cho thấy hơn 72% câu trả lời được đánh giá ở mức Tốt và Xuất sắc, khẳng định khả năng ứng dụng thực tế của hệ thống.

4.4.4. Phân tích trích dẫn(Citation Analysis)

Hệ thống được thiết kế để trích dẫn nguồn tài liệu tham khảo. Số liệu thống kê cho thấy:

- Recall trích dẫn: 0.708 (Hệ thống tìm được 70.8% các nguồn tài liệu đúng).
- Số lượng trích dẫn trung bình: 15 đoạn văn bản/câu trả lời.

Nhận xét: Hệ thống có xu hướng trích dẫn rộng (High Recall) để đảm bảo không bỏ sót thông tin, dẫn đến số lượng trích dẫn trung bình khá cao (15). Điều này làm giảm chỉ số Precision (0.047) và tăng tỷ lệ nhiễu (Hallucination Rate - theo nghĩa trích dẫn các đoạn không trực tiếp chứa đáp án). Đây là sự đánh đổi chấp nhận được để ưu tiên độ bao phủ thông tin (Recall) trong các bài toán tra cứu học thuật.

Chương 5: Kết luận và hướng phát triển

5.1 Kết luận

Nghiên cứu này đã xây dựng và triển khai thành công hệ thống Chatbot hỏi đáp (RAG Chatbot) chuyên dụng cho miền dữ liệu học thuật tiếng Việt, tập trung giải quyết bài toán truy xuất thông tin chính xác từ các tài liệu giáo trình và bài giảng.

Các kết quả đạt được có thể tóm tắt qua ba khía cạnh chính:

1. Xây dựng mô hình Reranker tối ưu cho tiếng Việt:

- Đã thực hiện tinh chỉnh (Fine-tuning) thành công mô hình `itdaib/PhoRanker` trên tập dữ liệu chuyên ngành.
- Kết quả thực nghiệm cho thấy mô hình đạt độ chính xác 83.06% trên tập kiểm thử, vượt trội so với các phương pháp tìm kiếm từ khóa truyền thống.
- Việc tích hợp Reranker vào quy trình Hybrid Search đã giúp hệ thống đạt chỉ số Recall@10 là 79.2%, đảm bảo trong gần 80

2. Hiệu năng sinh văn bản và trải nghiệm người dùng:

- Hệ thống sinh ra câu trả lời có độ chính xác nội dung cao (ROUGE-1 đạt 0.605) và được đánh giá tích cực bởi mô hình ngôn ngữ lớn (Gemini-as-a-Judge) với điểm số trung bình 4.04/5.0.
- Hơn 72% câu trả lời được đánh giá ở mức "Tốt" và "Xuất sắc", chứng tỏ hệ thống có khả năng ứng dụng thực tế cao hỗ trợ sinh viên ôn tập.

3. Hệ thống hoàn chỉnh và Minh bạch:

- Xây dựng thành công giao diện Web tích hợp đầy đủ các module: Quản lý dữ liệu, Truy xuất, Xếp hạng và Sinh câu trả lời.
- Tính năng Trích dẫn nguồn (Citation) hoạt động ổn định với độ bao phủ (Recall) trích dẫn lên tới 70.8%, giúp người dùng dễ dàng kiểm chứng thông tin, giảm thiểu rủi ro ảo giác (hallucination).

5.2 Hạn chế của đề tài

Bên cạnh những kết quả tích cực, dựa trên số liệu đánh giá chi tiết, hệ thống vẫn tồn tại một số hạn chế cần khắc phục:

- Độ chính xác của trích dẫn (Citation Precision) thấp:
 - Mặc dù hệ thống tìm được tài liệu đúng (Recall cao), nhưng số lượng trích

dẫn trung bình khá lớn (15 chunks/câu trả lời) dẫn đến chỉ số Precision chỉ đạt 0.047. Điều này có nghĩa là hệ thống đang "lấy dư" quá nhiều đoạn văn bản không cần thiết, có thể gây nhiễu cho người đọc khi muốn tra cứu chính xác vị trí thông tin.

- Độ trễ xử lý (Latency):
 - Quá trình Reranking, mặc dù cải thiện độ chính xác, nhưng làm tăng thời gian phản hồi (trung bình thêm 100-200ms). Trên phần cứng giới hạn (RTX 3050), việc xử lý đồng thời nhiều truy vấn phức tạp có thể gây tắc nghẽn.
- Phụ thuộc vào dữ liệu huấn luyện:
 - Mô hình Fine-tune hiện tại hoạt động tốt trên miền dữ liệu môn học cụ thể nhưng chưa được kiểm thử rộng rãi trên các môn học khác có tính chất khác biệt (ví dụ: các môn xã hội học so với các môn kỹ thuật).

5.3 Hướng phát triển trong tương lai

Để khắc phục các hạn chế trên và nâng cao chất lượng hệ thống, các hướng nghiên cứu và phát triển tiếp theo được đề xuất bao gồm:

1. Tối ưu hóa chiến lược chọn lọc văn bản (Context Selection):
 - Áp dụng cơ chế Dynamic Thresholding (Ngưỡng động) cho điểm số Reranking: Thay vì luôn lấy Top K kết quả, hệ thống chỉ giữ lại các đoạn văn bản có điểm tương đồng (similarity score) vượt qua một ngưỡng nhất định (ví dụ: > 0.8) để loại bỏ các thông tin nhiễu, từ đó nâng cao chỉ số Precision.
 - Sử dụng kỹ thuật Context Compression hoặc LLM-based Filtering để tóm tắt hoặc lọc lại nội dung các đoạn văn bản trước khi đưa vào mô hình sinh câu trả lời.
2. Cải thiện hiệu năng phần cứng:
 - Nghiên cứu áp dụng kỹ thuật Lượng tử hóa (Quantization) (chuyển mô hình về INT8 hoặc FP8) và Knowledge Distillation (chưng cất tri thức) để giảm kích thước mô hình Reranker, giúp tăng tốc độ xử lý mà không làm giảm đáng kể độ chính xác.
 - Triển khai cơ chế Caching (lưu đệm) cho các truy vấn phổ biến để giảm tải tính toán lặp lại.
3. Mở rộng phạm vi và kỹ thuật RAG nâng cao:
 - Thử nghiệm với GraphRAG (Sử dụng đồ thị tri thức): Giúp hệ thống hiểu

sâu hơn mối quan hệ giữa các khái niệm trong giáo trình, từ đó trả lời tốt hơn các câu hỏi dạng suy luận đa bước (multi-hop reasoning) mà phương pháp Vector Search hiện tại còn hạn chế.

- Mở rộng bộ dữ liệu huấn luyện sang đa dạng các môn học khác để đánh giá khả năng tổng quát hóa (Generalization) của mô hình PhoRanker đã tinh chỉnh.

Tài liệu tham khảo

- [1] L. et. al, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” <https://arxiv.org/abs/2005.11401>, 2020.
- [2] A. Kamsetty, “Retrieval augmented generation with huggingface transformers and ray - amog kamsetty từ nhóm anyscales,” <https://huggingface.co/blog/ray-rag>, 2021.
- [3] R. Merritt, “What is retrieval-augmented generation, aka rag?” <https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/>, 2025.
- [4] I. Belcic, “What is retrieval augmented generation (rag)?” <https://www.ibm.com/think/topics/retrieval-augmented-generation>, 2024.
- [5] S. Connelly, “Practical bm25 — part 2: The bm25 algorithm and its variables,” <https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables>, 2018.
- [6] I. G. Nils Reimers, “Sentence-bert: Sentence embeddings using siamese bert-networks,” <https://arxiv.org/abs/1908.10084>, 2019.
- [7] S. B. Gordon V. Cormack Charles L A Clarke, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” *SIGIR '09: Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, 2009.
- [8] N. T. et. al, “Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models,” <https://arxiv.org/abs/2104.08663>, 2021.
- [9] Sentence-BERT, “Sbert documentation,” <https://www.sbert.net/examples/applications/cross-encoder/README.html>, 2023.
- [10] A. T. N. Dat Quoc Nguyen, “Phobert: Pre-trained language models for vietnamese,” <https://arxiv.org/abs/2003.00744>, 2020.
- [11] D. N. Ba, “Phoranker: A cross-encoder model for vietnamese ranking tasks,” <https://huggingface.co/itdainb/PhoRanker>, 2024.
- [12] D. N. Ba, “Phoranker · models · dataloop,” https://dataloader.ai/library/model/itdainb_phoranker/, 2024.
- [13] L. G. et. al, “Precise zero-shot dense retrieval without relevance labels,” <https://arxiv.org/abs/2212.10496>, 2022.

- [14] S. E. et. al, “Ragas: Automated evaluation of retrieval augmented generation,” <https://arxiv.org/abs/2309.15217>, 2023.